

SOAP CON JAVA

1. ¿Qué significa SOAP?

- A. Simple Object Access Protocol.
- B. Service Oriented Architecture Protocol.
- C. Secure Object Access Protocol.
- D. Standard Object Access Protocol.

Justificación:

SOAP es el acrónimo de Simple Object Access Protocol, que en español significa Protocolo Simple de Acceso a Objetos. Es un protocolo de comunicación utilizado en servicios web para el intercambio de información estructurada entre diferentes sistemas.

2. ¿Cuál de las siguientes afirmaciones describe correctamente SOAP?

- A. Es un protocolo de comunicación basado en REST.
- B. Es un protocolo de comunicación basado en XML.
- C. Es un lenguaje de programación orientado a objetos.
- D. Es un formato de archivo para almacenar datos estructurados.

Justificación:

SOAP se basa en XML para estructurar los mensajes y establecer la comunicación entre aplicaciones.

3. ¿Cuál es el objetivo principal de SOAP?

- A. Transmitir datos en formato binario.
- B. Establecer conexiones seguras entre clientes y servidores.
- C. Facilitar la interoperabilidad entre diferentes plataformas.
- D. Realizar consultas y operaciones en bases de datos.

Justificación:

SOAP se diseñó para permitir la comunicación entre aplicaciones desarrolladas en diferentes lenguajes de programación y ejecutadas en diferentes plataformas.

4. ¿Qué tipo de mensajes se utilizan en SOAP?

- A. GET y POST
- B. XML y JSON
- C. HTML y CSS
- D. Request y Response

Justificación:

En SOAP, los mensajes se dividen en solicitudes (request) enviadas por el cliente y respuestas (response) enviadas por el servidor.

5. ¿Cuál es el formato de mensaje utilizado en SOAP?

- A. JSON
- B. XML
- C. HTML
- D. CSV

Justificación:

SOAP utiliza XML (Extensible Markup Language) para estructurar los mensajes y describir los datos que se intercambian entre las aplicaciones.

6. ¿Qué componente de Java se utiliza para implementar servicios web SOAP?

- A. JSP (JavaServer Pages)
- B. JDBC (Java Database Connectivity)
- C. JAX-WS (Java API for XML Web Services)**
- D. JSON (JavaScript Object Notation)

Justificación:

JAX-WS es una API de Java que proporciona las herramientas necesarias para desarrollar servicios web SOAP en Java.

7. ¿Cuál es el método HTTP más comúnmente utilizado para enviar mensajes SOAP?

- A. GET
- B. POST**
- C. PUT
- D. DELETE

Justificación:

Aunque SOAP no se basa en HTTP, el método POST es comúnmente utilizado para enviar mensajes SOAP a través de HTTP.

8. ¿Cuál es el beneficio principal de usar SOAP en comparación con otros protocolos de comunicación?

- A. Mayor velocidad de transferencia de datos.
- B. Mejor seguridad en la comunicación.
- C. Mayor soporte para diferentes formatos de datos.**
- D. Mayor eficiencia en el consumo de recursos.

Justificación:

SOAP es capaz de soportar una amplia variedad de formatos de datos, lo que lo hace flexible y adecuado para la interoperabilidad entre diferentes sistemas.

9. ¿Qué se utiliza para describir los servicios web SOAP?

- A. WSDL (Web Services Description Language)**
- B. XML Schema
- C. JSON Schema
- D. JavaDoc

Justificación:

WSDL se utiliza para describir los servicios web SOAP, incluyendo los métodos disponibles, los tipos de datos utilizados y las ubicaciones de los servicios.

10. ¿Cuál es el estándar de seguridad utilizado comúnmente en SOAP?

- A. SSL (Secure Sockets Layer)**
- B. JWT (JSON Web Tokens)
- C. OAuth (Open Authorization)
- D. HTTPS (Hypertext Transfer Protocol Secure)

Justificación:

SSL es un estándar de seguridad comúnmente utilizado para asegurar las comunicaciones SOAP y garantizar la confidencialidad y autenticidad de los mensajes.

11. ¿Cuál es la función principal de SOAP en Java?

- A. Comunicación entre diferentes sistemas operativos.
- B. Acceso a objetos remotos.**
- C. Creación de interfaces gráficas de usuario.
- D. Manejo de bases de datos.

Justificación:

SOAP permite la comunicación entre diferentes sistemas a través de la red y facilita el acceso a objetos remotos mediante la invocación de métodos y el intercambio de mensajes estructurados.

12. ¿Cuál de las siguientes afirmaciones es correcta sobre SOAP en Java?

- A. SOAP es un lenguaje de programación.
- B. SOAP se basa en el formato JSON.
- C. SOAP utiliza XML para el intercambio de datos.**
- D. SOAP solo es compatible con servidores web Apache.

Justificación:

SOAP utiliza XML (Extensible Markup Language) para estructurar los datos intercambiados entre los diferentes sistemas. XML proporciona un formato flexible y legible para el intercambio de información.

13. ¿Cuál de las siguientes tecnologías se utiliza comúnmente junto con SOAP en Java?

- A. JDBC
- B. RMI**
- C. REST
- D. Hibernate

Justificación:

SOAP se utiliza comúnmente en combinación con RMI (Remote Method Invocation) en Java para permitir el acceso y la invocación de métodos en objetos remotos a través de la red.

14. ¿Cuál es la ventaja principal de SOAP en Java en comparación con REST?

- A. Mayor rendimiento.
- B. Menor consumo de recursos.
- C. Mayor interoperabilidad.**
- D. Mayor simplicidad.

Justificación:

SOAP se considera más interoperable que REST, lo que significa que es más fácil para diferentes sistemas comunicarse entre sí debido a su estructura basada en XML y su capacidad para definir contratos de servicios más rigurosos.

15. ¿Cuál es el formato de mensaje utilizado en SOAP?

- A. JSON.
- B. XML.**
- C. HTML.
- D. TXT.

Justificación:

SOAP utiliza XML (Extensible Markup Language) como formato de mensaje para estructurar la información intercambiada entre los sistemas. XML proporciona una forma flexible y legible para representar datos en un formato compatible con diferentes plataformas y lenguajes de programación.

16. ¿Cuál de las siguientes tecnologías se utiliza comúnmente para implementar servicios web SOAP en Java?

- A. REST.
- B. JAX-RS.**
- C. JAX-WS.
- D. Servlets.

Justificación:

JAX-WS (Java API for XML Web Services) es la tecnología comúnmente utilizada para implementar servicios web SOAP en Java. Proporciona un conjunto de APIs y herramientas para desarrollar y desplegar servicios web basados en SOAP utilizando la plataforma Java.

17. ¿Cuál es la ventaja principal de usar SOAP en comparación con otros protocolos?

- A. Es más rápido.
- B. Es más seguro.
- C. Ofrece interoperabilidad entre diferentes plataformas.**
- D. Es más sencillo de implementar.

Justificación:

Una de las ventajas principales de SOAP es su capacidad para ofrecer interoperabilidad entre diferentes plataformas y lenguajes de programación. Debido a que SOAP utiliza XML como formato de mensaje, puede ser utilizado en sistemas heterogéneos, lo que permite la comunicación entre aplicaciones desarrolladas en distintas tecnologías. Esto facilita la integración y el intercambio de información entre sistemas distribuidos.

Si bien SOAP también puede brindar seguridad y soporte para transacciones distribuidas, estas características no son exclusivas de SOAP y también pueden ser proporcionadas por otros protocolos. Además, la afirmación de que SOAP es más rápido o más sencillo de implementar no es necesariamente cierta, ya que la velocidad y la facilidad de implementación dependen de varios factores, como el contexto de uso y los requisitos específicos del proyecto.

18. ¿Cuál es el componente central de una solicitud SOAP?

- A. Body.
- B. Header.**
- C. Envelope.
- D. Fault.

Justificación:

El componente central de una solicitud SOAP es el encabezado (header). El encabezado SOAP contiene información adicional sobre la solicitud, como detalles de autenticación, información de enrutamiento y cualquier otra información relevante para el procesamiento de la solicitud por parte del servicio web.

El encabezado SOAP es opcional y puede estar presente o ausente en una solicitud SOAP. Si está presente, proporciona datos adicionales que pueden ser utilizados por el servicio web para tomar decisiones sobre el procesamiento de la solicitud. Por ejemplo, el encabezado puede contener información de seguridad como tokens de autenticación o firma digital.

El cuerpo (body) de la solicitud SOAP, opción a), contiene los datos o la carga útil de la solicitud en sí. Representa la parte principal de la solicitud y contiene la información que se enviará al servicio web para su procesamiento.

El sobre (envelope) de SOAP, opción c), es el contenedor principal que envuelve tanto el encabezado como el cuerpo de la solicitud SOAP. Proporciona la estructura y el formato para enviar la solicitud correctamente y asegura que la solicitud sea interpretable por el servicio web.

La opción Fault se refiere a un elemento utilizado en una respuesta SOAP para indicar errores o excepciones que pueden haber ocurrido durante el procesamiento de la solicitud. No es un componente central de la solicitud SOAP.

19. ¿Cuál de las siguientes afirmaciones es verdadera sobre los servicios web SOAP?

- A. Pueden comunicarse solo entre aplicaciones Java.
- B. Utilizan exclusivamente el protocolo HTTP.
- C. Son independientes del lenguaje de programación y plataforma.
- D. No permiten la comunicación entre diferentes sistemas.

Justificación:

Los servicios web SOAP son independientes del lenguaje de programación y plataforma, lo que significa que pueden ser utilizados para la comunicación entre sistemas desarrollados en diferentes tecnologías. No están limitados a la comunicación entre aplicaciones Java solamente (opción a)).

Si bien SOAP puede utilizarse sobre el protocolo HTTP, no está limitado exclusivamente a este protocolo. SOAP puede ser utilizado sobre otros protocolos de transporte como SMTP, FTP, TCP/IP, entre otros. Por lo tanto, la afirmación de que los servicios web SOAP utilizan exclusivamente el protocolo HTTP (opción b)) no es verdadera.

La opción d) no es verdadera, ya que los servicios web SOAP permiten la comunicación entre diferentes sistemas. Gracias a su estructura basada en XML y su capacidad para definir reglas y estándares, SOAP facilita la interoperabilidad entre sistemas heterogéneos, lo que significa que diferentes sistemas pueden comunicarse y compartir información utilizando servicios web SOAP.

En resumen, la afirmación verdadera es que los servicios web SOAP son independientes del lenguaje de programación y plataforma, lo que les permite comunicarse entre diferentes sistemas desarrollados en distintas tecnologías.

20. ¿Cuál de las siguientes tecnologías se utiliza para definir la estructura y el formato de los mensajes SOAP?

- A. WSDL.
- B. XML.
- C. JSON.
- D. XSD.

Justificación:

XML (Extensible Markup Language) se utiliza para definir la estructura y el formato de los mensajes SOAP. XML proporciona una forma estándar y legible por máquinas para organizar y representar los datos en los mensajes SOAP.

21. ¿Cuál de las siguientes opciones describe mejor la función del elemento SOAP Fault?

- A. Proporcionar información adicional en el cuerpo del mensaje SOAP.
- B. Indicar que se produjo un error durante la ejecución del servicio.
- C. Establecer la dirección de destino del mensaje SOAP.
- D. Controlar el flujo de ejecución del servicio.

Justificación:

El elemento **SOAP Fault** es la estructura estándar que utiliza un servicio SOAP para **notificar que ocurrió un error** al procesar un mensaje. Su propósito principal no es dirigir mensajes ni controlar el flujo, sino **reportar información sobre una falla** (qué salió mal, por qué y —cuando procede— detalles técnicos para depuración).

Puntos clave:

- **Notificación de errores (función principal):** Cuando el receptor no puede procesar la solicitud (por ejemplo: operación inexistente, errores de validación, excepciones internas, problemas de seguridad, etc.), devuelve un mensaje SOAP cuyo cuerpo contiene un elemento `<Fault>` (o `<soap:Fault>`). Esto comunica al invocador que la operación falló y proporciona información sobre la causa. Por eso la opción **B** es la correcta.
- **Estructura y contenido:** El `Fault` incluye campos para el código del fallo y un texto legible; además puede incluir detalles técnicos. Por ejemplo:

SOAP 1.1 (ejemplo):

```
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring>Error interno en el servicio</faultstring>
      <faultactor>http://mi-servidor/servicio</faultactor>
      <detail>
        <errorDetail>Causa técnica: NullPointerException en
X</errorDetail>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

SOAP 1.2 (ejemplo):

```
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
  <env:Body>
    <env:Fault>
      <env:Code>
        <env:Value>env:Sender</env:Value>
      </env:Code>
      <env:Reason>
        <env:Text xml:lang="es">Solicitud inválida: falta parámetro
"id"</env:Text>
      </env:Reason>
      <env:Detail>
        <errorDetail>Parametro 'id' no encontrado</errorDetail>
      </env:Detail>
    </env:Fault>
  </env:Body>
</env:Envelope>
```

- **Por qué las otras opciones son incorrectas o incompletas:**
 - **A. Proporcionar información adicional en el cuerpo del mensaje SOAP.**
Aunque el `Fault` **se encuentra** en el cuerpo del mensaje y **sí** contiene información, su papel específico **no es** simplemente añadir datos adicionales; su propósito es **informar de un error**. La opción A es vaga e insuficiente como descripción principal.
 - **C. Establecer la dirección de destino del mensaje SOAP.**
La dirección de destino la determina el encabezado (por ejemplo, `To, Address`) o el transporte (`HTTP Host/URL`), no el `Fault`.
 - **D. Controlar el flujo de ejecución del servicio.**
El `Fault` comunica un estado (error) después de que algo ya falló; **no controla** la lógica de flujo ni la ejecución del servicio.

22. ¿Qué tipo de arquitectura de servicios web sigue el protocolo SOAP?

- A. RESTful.
- B. Service-Oriented Architecture (SOA).**
- C. Microservices.
- D. MVC.

Justificación:

El protocolo **SOAP (Simple Object Access Protocol)** se enmarca dentro de la **arquitectura orientada a servicios**, conocida como **SOA (Service-Oriented Architecture)**. Su diseño, estructura y propósito coinciden con los principios fundamentales de esta arquitectura, en la que **los servicios son componentes independientes que se comunican entre sí a través de mensajes estandarizados**.

La **Service-Oriented Architecture (SOA)** es un estilo de arquitectura de software en el que las funcionalidades de una aplicación se dividen en **servicios independientes**, los cuales:

- Se comunican mediante **mensajes XML o JSON**.
- Son **reutilizables, interoperables y desacoplados**.
- Pueden ejecutarse en **diferentes plataformas o lenguajes de programación**, pero deben comunicarse a través de **protocolos estándar**.

SOAP fue diseñado precisamente para cumplir estos principios.

¿Cómo se relaciona SOAP con SOA?

SOAP es un **protocolo de mensajería basado en XML** que permite la comunicación entre aplicaciones distribuidas.

Dentro de SOA, **SOAP actúa como el medio de comunicación** entre servicios, utilizando mensajes estructurados que contienen:

- Un **Envelope (sobre)** para delimitar el mensaje.
- Un **Header (encabezado)** con metadatos o información de control.
- Un **Body (cuerpo)** con los datos del mensaje o la solicitud.
- Un **Fault (error)** para reportar fallos.

SOAP trabaja habitualmente junto con **WSDL (Web Services Description Language)**, que describe las operaciones y datos del servicio, y con **UDDI (Universal Description, Discovery and Integration)**, que permite registrar y descubrir servicios. Estos tres elementos —**SOAP, WSDL y UDDI**— constituyen la base tecnológica de **SOA**.

¿Por qué las otras opciones son incorrectas?

- **A. RESTful:**
REST no es un protocolo, sino un **estilo arquitectónico alternativo** a SOAP. REST se basa en **HTTP y recursos identificados por URL**, mientras que SOAP usa **mensajes XML estructurados y más rígidos**.
→ SOAP ≠ RESTful.
- **C. Microservices:**
Aunque los microservicios **también son servicios independientes**, se comunican típicamente mediante **HTTP/JSON o mensajería ligera**, no SOAP. SOAP pertenece a una **generación anterior** de diseño orientado a servicios (SOA), no a la arquitectura moderna de microservicios.
- **D. MVC (Modelo-Vista-Controlador):**
MVC es un **patrón de diseño** usado para estructurar aplicaciones (especialmente interfaces gráficas o web), **no** un tipo de arquitectura de servicios web.

23. ¿Cuál de las siguientes opciones describe mejor el propósito del elemento SOAP Header?

- A. Contiene la información principal del mensaje SOAP.
- B. Es utilizado para autenticar la solicitud SOAP.
- C. **Proporciona información adicional y opcional sobre la solicitud o respuesta SOAP.**
- D. Define las reglas de validación para el mensaje SOAP.

Justificación:

El **SOAP Header** es un elemento opcional dentro del `Envelope` que **transporta metadatos y extensiones** relacionados con el mensaje SOAP. Su propósito principal **no es** contener la "información principal" (esa va en el `Body`), sino aportar **información adicional, de control o contextual** que puede ser usada por el receptor o por intermediarios en la ruta del mensaje.

Funciones típicas del SOAP Header

- **Autenticación y seguridad:** encabezados como los de **WS-Security** (tokens, firmas, timestamps) se colocan en el `Header`.
- **Enrutamiento y control de intermediarios:** instrucciones para intermediarios, dirección de rol/actor, o información de procesamiento.
- **Transacciones y calidad de servicio:** identificadores de transacción, prioridad, límite de tiempo (expiry), etc.
- **Contexto de sesión o correlación:** IDs para correlacionar peticiones y respuestas entre sistemas distribuidos.
- **Extensiones específicas:** cualquier extensión definida por la aplicación o por estándares que necesiten metadatos separados del cuerpo.

Características importantes

- **Opcionalidad:** el `Header` puede no existir; sólo el `Body` es necesario para transportar la operación.
- **Segmentación en bloques:** puede contener varios elementos hijos, cada uno con su propósito y destinatario.
- **Atributo `mustUnderstand`:** permite indicar si un receptor **debe** procesar un bloque del header; si no lo entiende y `mustUnderstand="1"`, debe devolver un `Fault`.
- **Atributo `actor / role`:** define qué nodo (intermediario o receptor final) debe procesar ese header.

Ejemplo (simplificado)

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <wsse:Security mustUnderstand="1"
      xmlns:wsse="http://schemas.xmlsoap.org/ws/2002/12/secext">
      <wsse:UsernameToken>
        <wsse:Username>usuario</wsse:Username>
        <wsse:Password>clave</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
    <m:Transaction xmlns:m="http://ejemplo.org/tx"
      soap:actor="http://schemas.xmlsoap.org/soap/actor/next">
      <m:TxId>12345</m:TxId>
    </m:Transaction>
  </soap:Header>
  <soap:Body>
    <!-- Información principal / operación -->
  </soap:Body>
</soap:Envelope>
```

Por qué las otras opciones son incorrectas o incompletas

- **A. Contiene la información principal del mensaje SOAP.**
Incorrecto: la información principal (la carga útil de la operación) se pone en el Body. El Header contiene metadatos/controles, no la carga funcional principal.
- **B. Es utilizado para autenticar la solicitud SOAP.**
Parcialmente cierto pero **engñoso**: el header **puede** usarse para autenticación (ej. WS-Security), pero **esa no es su única ni exclusiva** función. La opción C describe el propósito general.
- **D. Define las reglas de validación para el mensaje SOAP.**
Incorrecto: las reglas de validación (esquemas XSD, WSDL, políticas) se definen fuera del header; el header no es el mecanismo para declarar validaciones globales del mensaje.

24. ¿Cuál de las siguientes tecnologías se utiliza comúnmente para implementar servicios web SOAP en Java?

- A. JAX-WS.**
- B. Spring MVC.
- C. Java Servlets.
- D. Java Persistence API (JPA).

Justificación:

La tecnología **JAX-WS** (Java API for XML Web Services) es la **API estándar de Java** para **crear, implementar y consumir servicios web basados en SOAP**. Forma parte de la plataforma **Jakarta EE** (antes **Java EE**) y también puede usarse en aplicaciones **Java SE**.

Su propósito es **facilitar la comunicación entre aplicaciones distribuidas** usando el protocolo **SOAP** y los estándares relacionados (**WSDL, XML Schema, HTTP, UDDI**, etc.), sin que el programador tenga que manejar manualmente los mensajes XML.

Características principales de JAX-WS

1. **Basada en SOAP y WSDL:**
 - Los mensajes se codifican en formato **XML** según las reglas de SOAP.
 - El servicio se describe mediante **WSDL (Web Services Description Language)**.
2. **Generación automática de código:**
 - Puede generar el código Java a partir de un archivo **WSDL** (cliente o servidor).
 - También puede generar el **WSDL automáticamente** a partir de clases Java anotadas.

3. **Uso de anotaciones:**

Facilita la creación de servicios con anotaciones como:

```
import javax.jws.WebService;  
import javax.jws.WebMethod;  
  
@WebService  
public class Calculadora {  
    @WebMethod  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
}
```

Este servicio automáticamente expone un endpoint SOAP accesible, por ejemplo:
<http://localhost:8080/CalculadoraService?wsdl>

4. **Interoperabilidad:**

- Permite la comunicación entre sistemas **Java y no Java**, gracias a que SOAP y WSDL son estándares abiertos.

Por qué las otras opciones son incorrectas

- **B. Spring MVC:**

Es un **framework** para aplicaciones web basadas en el patrón **Modelo-Vista-Controlador (MVC)**.

Aunque Spring puede trabajar con servicios REST o SOAP, **Spring MVC** está orientado a aplicaciones web RESTful o interfaces de usuario, **no específicamente a SOAP**.

- **C. Java Servlets:**

Los Servlets son la **base** sobre la que se ejecutan muchos frameworks web en Java, pero **no son una tecnología de servicios SOAP por sí mismos**. JAX-WS puede implementarse **sobre Servlets**, pero éstos solo gestionan el transporte HTTP, **no el protocolo SOAP ni el procesamiento XML**.

- **D. Java Persistence API (JPA):**

JPA se utiliza para la **persistencia de datos en bases de datos relacionales**, **no** para comunicación entre servicios web. Puede complementarse con JAX-WS en una aplicación, pero no sirve para implementar servicios SOAP.

25. ¿Qué es WSDL en el contexto de SOAP?

- A. **Web Services Definition Language.**
- B. Web Services Development Library.
- C. Web Services Data Language.
- D. Web Services Description Logic.

Justificación:

El término **WSDL** significa **Web Services Definition Language**, y es un **lenguaje basado en XML** utilizado para **describir formalmente un servicio web SOAP**: qué operaciones ofrece, qué mensajes intercambia, qué tipos de datos utiliza y cómo acceder a él.

En otras palabras, **WSDL actúa como el “contrato” o descripción técnica del servicio web**, permitiendo que cualquier cliente (sin importar el lenguaje o plataforma) sepa **cómo comunicarse correctamente** con el servicio.

Características principales de WSDL

1. **Basado en XML:**

WSDL utiliza XML para describir todos los aspectos del servicio, garantizando **interoperabilidad** entre sistemas heterogéneos.

2. **Describe completamente el servicio SOAP:**

Incluye información sobre:

- **Operaciones** disponibles (qué funciones ofrece el servicio).
- **Mensajes de entrada y salida** (qué datos espera y qué devuelve).
- **Tipos de datos** utilizados (definidos normalmente con XML Schema).
- **Protocolo de transporte** (por ejemplo, HTTP, SMTP, etc.).
- **Dirección o endpoint** donde se encuentra el servicio.

3. **Rol dentro del ecosistema SOAP:**

- **SOAP define cómo se envían los mensajes.**
- **WSDL define qué operaciones existen y cómo deben estructurarse los mensajes SOAP.**
- **Juntos, permiten que el cliente y el servidor se entiendan sin ambigüedad.**
-

Ejemplo simplificado de WSDL

```
<definitions name="CalculadoraService"
  targetNamespace="http://ejemplo.org/calculadora"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://ejemplo.org/calculadora"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://schemas.xmlsoap.org/wsdl/">

  <message name="SumarRequest">
    <part name="a" type="xsd:int"/>
    <part name="b" type="xsd:int"/>
  </message>

  <message name="SumarResponse">
    <part name="resultado" type="xsd:int"/>
  </message>

  <portType name="CalculadoraPortType">
    <operation name="sumar">
      <input message="tns:SumarRequest"/>
      <output message="tns:SumarResponse"/>
    </operation>
  </portType>
```

```
<binding name="CalculadoraBinding" type="tns:CalculadoraPortType">
  <soap:binding style="rpc"
transport="http://schemas.xmlsoap.org/soap/http"/>
</binding>

<service name="CalculadoraService">
  <port name="CalculadoraPort" binding="tns:CalculadoraBinding">
    <soap:address location="http://localhost:8080/CalculadoraService"/>
  </port>
</service>
</definitions>
```

Este documento permite a un cliente generar automáticamente el código necesario para invocar el servicio (por ejemplo, usando **JAX-WS** en Java).

Por qué las otras opciones son incorrectas

- **B. Web Services Development Library:**

No es una biblioteca de desarrollo; WSDL **no contiene código**, sino **una descripción XML** del servicio.

- **C. Web Services Data Language:**

Incorrecto. Aunque maneja datos, su propósito no es definir estructuras de datos, sino **describir servicios** (operaciones, mensajes y protocolos).

- **D. Web Services Description Logic:**

Tampoco. “Description Logic” se refiere a un concepto de la inteligencia artificial (ontologías), **no** a la descripción de servicios web.

26. ¿Cuál de las siguientes afirmaciones es verdadera sobre SOAP?

- A. Es un protocolo de comunicación que solo permite la transferencia de datos en formato XML.
- B. SOAP no es compatible con servicios web basados en REST.
- C. **SOAP utiliza el protocolo HTTP para la transferencia de mensajes.**
- D. SOAP solo se puede utilizar en aplicaciones Java.

Justificación:

La afirmación **C** es la más adecuada porque **HTTP es el transporte más usado** para enviar y recibir mensajes SOAP —es el binding más habitual— y los ejemplos, tutoriales y herramientas normalmente asumen HTTP (o HTTPS) como transporte. Un típico intercambio SOAP se realiza sobre HTTP mediante peticiones POST que llevan el *Envelope* XML en el cuerpo y un encabezado `Content-Type: text/xml o application/soap+xml`. **Pero importante:** esto no significa que SOAP *exclusivamente* use HTTP; SOAP está diseñado para ser **agnóstico al transporte** y puede funcionar sobre SMTP, JMS, TCP u otros protocolos (SOAP 1.2 formaliza bindings alternativos). La afirmación C es correcta en el sentido práctico y formativo: **sí utiliza HTTP**.

Por qué las otras opciones son incorrectas

- **A. "Es un protocolo de comunicación que solo permite la transferencia de datos en formato XML."**

Parcialmente engañosa. El cuerpo de un mensaje SOAP (el *Envelope*) **está definido en XML**, por lo que SOAP **usa XML** para la estructura del mensaje. Sin embargo, decir “*solo permite*” es impreciso: existen mecanismos para enviar datos binarios (p. ej. **MIME attachments, MTOM**) y bindings para otros formatos en escenarios

especiales. Además, SOAP puede transportarse por protocolos que admiten distintos tipos de contenido.

⇒ **Falso** por la palabra “solo”.

- **B. "SOAP no es compatible con servicios web basados en REST."**
Incorrecto: SOAP y REST son **diferentes estilos** (SOAP es un protocolo/estándar de mensajería con contrato WSDL; REST es un estilo arquitectónico orientado a recursos y a las operaciones HTTP estándar). Decir que “no son compatibles” es confuso: **no son lo mismo** y generalmente no se mezclan en diseño, pero técnicamente pueden coexistir en la misma arquitectura y pueden interoperar vía adaptadores/puentes. Además, nada impide que un cliente HTTP interactúe con un endpoint SOAP (aunque no siga los principios REST).
⇒ **Falso** (la afirmación es demasiado taxativa y equivocada).
- **D. "SOAP solo se puede utilizar en aplicaciones Java."**
Totalmente falso. SOAP es **independiente de plataforma y lenguaje**: existen implementaciones en Java (JAX-WS), .NET (WCF/ASMX), Python (zeep), PHP (ext-soap), Ruby, etc. El objetivo de SOAP es precisamente la **interoperabilidad entre plataformas**.
⇒ **Falso**.

Ejemplo breve (SOAP sobre HTTP)

Petición HTTP POST típica:

```
POST /ServicioCalculadora HTTP/1.1
Host: ejemplo.com
Content-Type: text/xml; charset=utf-8
Content-Length: 245
```

```
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <m:sumar xmlns:m="http://ejemplo.org/calculadora">
      <m:a>2</m:a>
      <m:b>3</m:b>
    </m:sumar>
  </soap:Body>
</soap:Envelope>
```

27. ¿Cuál de las siguientes opciones describe mejor la función del elemento SOAP Body?

- A. Contiene la información principal del mensaje SOAP.**
- B. Define la dirección de destino del mensaje SOAP.
- C. Proporciona información adicional sobre la solicitud o respuesta SOAP.
- D. Establece las reglas de validación para el mensaje SOAP.

Justificación:

El elemento **SOAP Body** es la **parte central y obligatoria** del mensaje SOAP. Su función principal es **transportar la información o los datos reales del mensaje**, es decir, la **solicitud o la respuesta** que se intercambia entre el cliente y el servicio web.

En otras palabras, el **Body** contiene el **contenido funcional del mensaje**, mientras que el Header lleva información opcional o de control (como seguridad, sesión, o transacciones).

Estructura general de un mensaje SOAP

Un mensaje SOAP tiene esta estructura básica:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <!-- Información opcional: seguridad, transacciones, etc. -->
  </soap:Header>

  <soap:Body>
    <!-- Información principal (operación y parámetros) -->
    <m:sumar xmlns:m="http://ejemplo.org/calculadora">
      <m:a>5</m:a>
      <m:b>3</m:b>
    </m:sumar>
  </soap:Body>
</soap:Envelope>
```

En este ejemplo:

- El elemento <soap:Body> contiene la operación sumar y sus parámetros.
- Esa información **representa la solicitud** que el cliente envía al servicio.
- En la respuesta, el Body contendría el **resultado**.

Características importantes del SOAP Body

1. **Obligatorio:**
Todo mensaje SOAP **debe** incluir un elemento Body.
2. **Contiene los datos principales:**
En una solicitud: los **parámetros** de la operación.
En una respuesta: los **resultados** o un posible Fault.
3. **Lugar donde se reportan errores:**
Si ocurre un error, el Body contiene el elemento <Fault>, que describe el problema.
4. **Procesado por el destino final:**
A diferencia del Header, que puede ser procesado por intermediarios, el Body **solo es interpretado por el receptor final** del mensaje.

Por qué las otras opciones son incorrectas

- **B. Define la dirección de destino del mensaje SOAP.**
✗ Incorrecto. La dirección o destino se define en los encabezados del mensaje HTTP o en elementos del **Header** (como soap:actor o soap:role), **no en el Body**.
- **C. Proporciona información adicional sobre la solicitud o respuesta SOAP.**
✗ Incorrecto. Esa es la función del **Header**, que transporta información **adicional u opcional** (por ejemplo, seguridad o autenticación).
- **D. Establece las reglas de validación para el mensaje SOAP.**
✗ Incorrecto. Las reglas de validación se definen en **WSDL o XML Schema (XSD)**, no dentro del Body.

28. ¿Qué significa el acrónimo WSDL?

- A. **Web Services Description Language.**
- B. Web Services Definition Logic.
- C. Web Services Development Library.
- D. Web Services Data Language.

Justificación:

WSDL significa **Web Services Description Language**, es decir, **Lenguaje de Descripción de Servicios Web**.

Se trata de un **lenguaje basado en XML** que sirve para **describir de manera formal y detallada un servicio web**, especificando:

- **Qué operaciones** ofrece el servicio.
- **Qué mensajes** (entradas y salidas) se intercambian.
- **Qué tipos de datos** se utilizan (definidos con XML Schema).
- **Qué protocolo y dirección (endpoint)** se deben usar para acceder al servicio.

Por tanto, **WSDL actúa como un contrato** entre el **proveedor** y el **consumidor** de un servicio web, permitiendo la **interoperabilidad entre sistemas de distintas plataformas y lenguajes**.

Ejemplo simplificado de WSDL

```
<definitions name="CalculadoraService"
  targetNamespace="http://ejemplo.org/calculadora"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://ejemplo.org/calculadora"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://schemas.xmlsoap.org/wsdl/">

  <message name="SumarRequest">
    <part name="a" type="xsd:int"/>
    <part name="b" type="xsd:int"/>
  </message>

  <message name="SumarResponse">
    <part name="resultado" type="xsd:int"/>
  </message>

  <portType name="CalculadoraPortType">
    <operation name="sumar">
      <input message="tns:SumarRequest"/>
      <output message="tns:SumarResponse"/>
    </operation>
  </portType>

  <binding name="CalculadoraBinding" type="tns:CalculadoraPortType">
    <soap:binding style="rpc" transport="http://schemas.xmlsoap.org/soap/http"/>
  </binding>

  <service name="CalculadoraService">
    <port name="CalculadoraPort" binding="tns:CalculadoraBinding">
      <soap:address location="http://localhost:8080/CalculadoraService"/>
    </port>
  </service>
</definitions>
```

Este documento le dice a cualquier cliente **cómo conectarse y qué formato de mensaje** usar para invocar el servicio sumar.

Por qué las otras opciones son incorrectas

- **B. Web Services Definition Logic:**
✗ Incorrecto. “Definition Logic” no existe en el contexto de servicios web. WSDL no define lógica de negocio, sino la **descripción técnica** del servicio.
- **C. Web Services Development Library:**
✗ Incorrecto. WSDL no es una biblioteca de desarrollo, sino un **documento XML descriptivo**.
- **D. Web Services Data Language:**
✗ Incorrecto. Aunque maneja datos en su descripción, su objetivo no es definir datos, sino **describir servicios web**.

29. ¿Cuál de las siguientes opciones describe mejor la función de la especificación JAX-WS en el contexto de SOAP?

- A. Es una especificación para el desarrollo de servicios web RESTful.
- B. Proporciona una API para el desarrollo de servicios web SOAP en Java.**
- C. Define el formato y la estructura de los mensajes SOAP.
- D. Establece las reglas de validación para los mensajes SOAP.

Justificación:

La especificación **JAX-WS (Java API for XML Web Services)** es la **API estándar de Java** que permite **crear, publicar y consumir servicios web basados en SOAP** de forma sencilla. Perteneció a la familia de especificaciones de **Jakarta EE (antes Java EE)** y está diseñada para manejar la comunicación mediante mensajes **SOAP** y descripciones **WSDL**.

Funciones principales de JAX-WS

1. **Facilitar el desarrollo de servicios SOAP**
JAX-WS abstrae la complejidad del manejo de mensajes XML y SOAP, ofreciendo una API orientada a objetos. El programador puede centrarse en la lógica del servicio sin preocuparse por construir o interpretar manualmente los mensajes SOAP.
2. **Generación automática de WSDL y clases cliente**
 - Desde una clase Java anotada, JAX-WS puede **generar automáticamente el archivo WSDL**.
 - O, a partir de un WSDL existente, puede **generar el código cliente Java** usando la herramienta wsimport.

3. **Uso de anotaciones Java**

Ejemplo básico de un servicio JAX-WS:

```
import javax.jws.WebService;  
import javax.jws.WebMethod;  
  
@WebService  
public class Calculadora {  
    @WebMethod  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
}
```

Al ejecutar este servicio en un contenedor compatible (por ejemplo, GlassFish o Payara), se publica un endpoint SOAP accesible en:

<http://localhost:8080/CalculadoraService?wsdl>

4. **Interoperabilidad**

Los servicios desarrollados con JAX-WS pueden comunicarse con clientes de otras plataformas (.NET, Python, PHP, etc.) gracias a que SOAP y WSDL son **estándares abiertos**.

Por qué las otras opciones son incorrectas

- **A. Es una especificación para el desarrollo de servicios web RESTful.**
✗ Incorrecto. La especificación para servicios **RESTful** en Java es **JAX-RS (Java API for RESTful Web Services)**, no **JAX-WS**. JAX-WS trabaja exclusivamente con **SOAP**.
- **C. Define el formato y la estructura de los mensajes SOAP.**
✗ Incorrecto. El formato de los mensajes SOAP está definido por la **especificación SOAP** del **W3C**, no por **JAX-WS**. JAX-WS solo **utiliza** ese formato para enviar y recibir mensajes.
- **D. Establece las reglas de validación para los mensajes SOAP.**
✗ Incorrecto. Las validaciones de estructura o contenido se definen mediante **XML Schema (XSD)** y **WSDL**, no por JAX-WS.

30. ¿Cuál de las siguientes opciones describe mejor la relación entre SOAP y XML?

- A. SOAP es un formato de mensaje basado en XML.**
- B. XML es un formato de mensaje basado en SOAP.
- C. SOAP y XML son protocolos de comunicación diferentes pero complementarios.
- D. SOAP y XML son términos intercambiables y se refieren a lo mismo.

Justificación:

SOAP (Simple Object Access Protocol) es un **protocolo de mensajería** utilizado para el intercambio de información estructurada entre aplicaciones distribuidas, especialmente en servicios web. La característica más importante de SOAP es que **sus mensajes se codifican en formato XML (Extensible Markup Language)**.

Esto significa que **SOAP utiliza XML como su lenguaje de estructura y transporte de datos**.

Relación entre SOAP y XML

1. **SOAP usa XML como base para sus mensajes**
Cada mensaje SOAP está escrito en XML y sigue una estructura específica definida por la especificación del protocolo.
Esto incluye un **Envelope**, un **Header** opcional y un **Body** obligatorio.
2. **Estructura típica de un mensaje SOAP**

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <auth:token xmlns:auth="http://example.com/auth">abc123</auth:token>
  </soap:Header>
  <soap:Body>
    <m:getProducto xmlns:m="http://example.com/tienda">
      <m:id>25</m:id>
    </m:getProducto>
  </soap:Body>
</soap:Envelope>
```

Como se puede ver, **todo el mensaje SOAP está estructurado en XML**, lo que permite que diferentes plataformas interpreten y procesen los datos de manera uniforme.

3. **Ventajas de usar XML en SOAP**

- **Estandarización:** XML es un estándar del W3C, ampliamente soportado.
- **Interoperabilidad:** SOAP puede funcionar entre sistemas heterogéneos (Java, .NET, Python, etc.).
- **Extensibilidad:** XML permite añadir nuevas etiquetas sin romper la compatibilidad.

Por qué las demás opciones son incorrectas

- **B. XML es un formato de mensaje basado en SOAP.**
✗ Incorrecto. Es al revés: **SOAP está basado en XML**, no XML en SOAP.
- **C. SOAP y XML son protocolos de comunicación diferentes pero complementarios.**
✗ Incorrecto. XML **no es un protocolo de comunicación**, sino un **lenguaje de marcado**. SOAP **sí es un protocolo**, que **usa XML como su formato de mensajes**.
- **D. SOAP y XML son términos intercambiables y se refieren a lo mismo.**
✗ Falso. Aunque SOAP utiliza XML, **no son lo mismo**:
 - XML = lenguaje para estructurar datos.
 - SOAP = protocolo que usa XML para enviar esos datos.

31. **¿Cuál de las siguientes opciones describe mejor el enfoque de "contrato primero" en el desarrollo de servicios web SOAP?**

- A. Se implementa el servicio web y luego se define su contrato.
- B. Se define el contrato del servicio web antes de su implementación.**
- C. No es aplicable en el desarrollo de servicios web SOAP.
- D. El enfoque de "contrato primero" no se utiliza en SOAP.

Justificación:

32. **¿Cuál de las siguientes opciones describe mejor la diferencia entre SOAP y REST?**

- A. SOAP utiliza solo el protocolo HTTP, mientras que REST puede utilizar varios protocolos.
- B. SOAP se basa en XML para la transferencia de datos, mientras que REST puede utilizar varios formatos como JSON y XML.
- C. SOAP es más adecuado para aplicaciones empresariales complejas, mientras que REST es más simple y liviano.**
- D. SOAP y REST son términos intercambiables y se refieren a lo mismo.

Justificación:

En el contexto de los **servicios web SOAP**, el enfoque de **"contrato primero" (contract-first)** significa que **primero se define el contrato del servicio**, es decir, **la interfaz pública que describe qué operaciones ofrece el servicio, qué mensajes intercambia y cómo se estructuran esos mensajes**. Este contrato se especifica utilizando el estándar **WSDL (Web Services Description Language)**. Solo después de definir este contrato, los desarrolladores proceden a **implementar el código del servicio** que cumple con las especificaciones del WSDL.

Cómo funciona el enfoque "contrato primero"

1. **Paso 1: Definición del contrato (WSDL)**

Se crea un archivo .wsdl que describe:

- Las operaciones disponibles (métodos o funciones del servicio).
- Los tipos de datos que se intercambian.

- Los protocolos de comunicación (SOAP, HTTP, etc.).
- Las ubicaciones o endpoints del servicio.

Ejemplo básico de un contrato WSDL:

```
<definitions name="CalculadoraService"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://example.com/calculadora"
  targetNamespace="http://example.com/calculadora">

  <message name="SumaRequest">
    <part name="a" type="xsd:int"/>
    <part name="b" type="xsd:int"/>
  </message>

  <message name="SumaResponse">
    <part name="resultado" type="xsd:int"/>
  </message>

  <portType name="CalculadoraPortType">
    <operation name="sumar">
      <input message="tns:SumaRequest"/>
      <output message="tns:SumaResponse"/>
    </operation>
  </portType>
</definitions>
```

2. Paso 2: Generación del código

A partir del archivo WSDL, se pueden generar automáticamente las clases Java necesarias (tanto del lado **servidor** como **cliente**) utilizando herramientas como:

- wsimport (para clientes)
- wsgen (para servidores)

Estas herramientas crean los stubs y bindings que cumplen con el contrato.

3. Paso 3: Implementación del servicio

Luego, el desarrollador implementa la lógica del servicio asegurando que respete el contrato definido.

Ventajas del enfoque "contrato primero"

- **Interoperabilidad:** Al definir el contrato con WSDL desde el inicio, cualquier cliente (Java, .NET, Python, PHP, etc.) puede comunicarse con el servicio.
- **Claridad y estandarización:** El contrato establece claramente qué hace el servicio y cómo debe usarse.
- **Menor acoplamiento:** Los clientes y el servidor se basan en el mismo contrato, sin depender de la implementación interna.
- **Facilita la validación y pruebas:** Los mensajes SOAP pueden validarse fácilmente contra el esquema definido en el WSDL.

Por qué las demás opciones son incorrectas

- **A. Se implementa el servicio web y luego se define su contrato.**
✗ Incorrecto. Ese es el enfoque "**contract-last**", donde primero se crea el código y luego se genera el WSDL automáticamente.
SOAP **soporta ambos enfoques**, pero "**contrato primero**" es más riguroso y recomendado.

- **C. No es aplicable en el desarrollo de servicios web SOAP.**
✗ Falso. SOAP fue precisamente una de las primeras tecnologías en usar el concepto de “contrato primero”.
- **D. El enfoque de "contrato primero" no se utiliza en SOAP.**
✗ Incorrecto. Es muy utilizado en SOAP, especialmente en entornos empresariales donde se requiere una definición formal antes del desarrollo.

33. ¿Cuál de las siguientes opciones describe mejor la función del elemento SOAP Envelope?

- A. Define la estructura y el formato de los mensajes SOAP.
- B. Proporciona información adicional en el cuerpo del mensaje SOAP.
- C. Envuelve el mensaje SOAP completo y lo identifica como tal.**
- D. Contiene la información principal del mensaje SOAP.

Justificación:

El elemento **<Envelope>** es el **elemento raíz obligatorio** en todo mensaje SOAP. Su función principal es **encapsular (envolver) todo el contenido del mensaje SOAP**, indicando que se trata efectivamente de un **mensaje conforme al protocolo SOAP**.

En otras palabras, el **Envelope** es la “envoltura” que contiene todos los demás elementos del mensaje (como el **Header** y el **Body**) y le da al mensaje su identidad como un mensaje SOAP válido.

Estructura general de un mensaje SOAP

Un mensaje SOAP bien formado siempre debe tener esta estructura:

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <auth:token xmlns:auth="http://example.com/auth">abc123</auth:token>
  </soap:Header>

  <soap:Body>
    <m:getProducto xmlns:m="http://example.com/tienda">
      <m:id>25</m:id>
    </m:getProducto>
  </soap:Body>
</soap:Envelope>
```

◆ El **<soap:Envelope>**:

- Es el **contenedor raíz** del mensaje SOAP.
- Contiene **dos partes principales**:
 - **<Header>** (*opcional*) → información adicional, como autenticación, transacciones, etc.
 - **<Body>** (*obligatorio*) → contiene los datos reales del mensaje, como las solicitudes o respuestas del servicio.
- Declara el **espacio de nombres (namespace)** correspondiente a la versión SOAP utilizada:
 - **SOAP 1.1** → <http://schemas.xmlsoap.org/soap/envelope/>
 - **SOAP 1.2** → <http://www.w3.org/2003/05/soap-envelope>

Función esencial del Envelope

- **Identificar el mensaje como SOAP**
Sin el Envelope, el documento XML no sería reconocido como un mensaje SOAP válido.

- **Estructurar el contenido**

Define la **jerarquía básica** del mensaje (Header y Body).

Esto asegura que cualquier aplicación compatible con SOAP pueda **interpretar, procesar o validar** el mensaje correctamente.

Por qué las demás opciones son incorrectas

- **A. Define la estructura y el formato de los mensajes SOAP.**

✗ Parcialmente cierto, pero **no es función del Envelope definir la estructura**, sino **representarla**.

La estructura del mensaje SOAP está **definida por la especificación SOAP**, no por el elemento Envelope en sí.

- **B. Proporciona información adicional en el cuerpo del mensaje SOAP.**

✗ Incorrecto. La información adicional se incluye en el **Header**, no en el Envelope.

- **D. Contiene la información principal del mensaje SOAP.**

✗ Incorrecto. La información principal (como la solicitud o respuesta) se encuentra dentro del **Body**, no directamente en el Envelope.

34. ¿Cuál de las siguientes afirmaciones es verdadera sobre SOAP y seguridad?

A. SOAP no es compatible con mecanismos de seguridad, como la encriptación y la autenticación.

B. SOAP tiene un alto nivel de seguridad incorporado mediante la firma y el cifrado de mensajes.

C. SOAP solo se puede utilizar en redes privadas y no es seguro para su uso en Internet.

D. SOAP no permite la autenticación de usuarios en servicios web.

Justificación:

Una de las grandes ventajas de **SOAP (Simple Object Access Protocol)** frente a otros enfoques (como REST) es que **SOAP incluye soporte para estándares avanzados de seguridad**, lo que lo hace ideal para entornos **empresariales, bancarios o gubernamentales** donde la **confidencialidad, integridad y autenticación** son críticas.

SOAP logra esto gracias a la especificación **WS-Security (Web Services Security)**, un conjunto de **extensiones al protocolo SOAP** que permiten aplicar **mecanismos de seguridad directamente a nivel de mensaje**.

Principales características de seguridad en SOAP (WS-Security)

1. Firma digital (Digital Signature)

- Permite **garantizar la integridad y autenticidad** de los mensajes SOAP.
- Usa certificados digitales (X.509) para firmar partes del mensaje (por ejemplo, el Header o el Body).
- Si el mensaje es alterado, la firma deja de ser válida.

2. Cifrado de mensajes (Message Encryption)

- SOAP puede **encriptar total o parcialmente** el contenido del mensaje, asegurando la **confidencialidad** de la información transmitida.
- Esto es especialmente importante cuando se intercambian datos sensibles (por ejemplo, información médica o financiera).

3. Autenticación y Tokens de seguridad

- WS-Security permite incluir en el Header elementos de autenticación, como:
 - **UsernameToken** (usuario y contraseña),
 - **BinarySecurityToken** (certificados digitales),
 - **SAML Tokens** (Security Assertion Markup Language).

- Esto permite que los servicios SOAP autentiquen a los usuarios antes de procesar sus solicitudes.
- 4. **Seguridad independiente del transporte**
 - A diferencia del uso de HTTPS (que protege solo durante la transmisión), WS-Security protege los **mensajes mismos**, incluso si pasan por múltiples intermediarios.
 - Esto se conoce como **seguridad a nivel de mensaje (message-level security)**.

Ejemplo simplificado de un Header SOAP con WS-Security

```
<soap:Header>
  <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/">
    <wsse:UsernameToken>
      <wsse:Username>usuario123</wsse:Username>
      <wsse:Password>claveSegura</wsse:Password>
    </wsse:UsernameToken>
  </wsse:Security>
</soap:Header>
```

Este bloque permite **autenticar** la solicitud SOAP antes de que el servidor procese el mensaje.

Por qué las demás opciones son incorrectas

- **A. SOAP no es compatible con mecanismos de seguridad, como la encriptación y la autenticación.**
✗ Falso. SOAP **sí es compatible**, mediante **WS-Security**, con cifrado, firma y autenticación avanzada.
- **C. SOAP solo se puede utilizar en redes privadas y no es seguro para su uso en Internet.**
✗ Incorrecto. SOAP **es seguro para Internet** cuando se implementa con WS-Security o HTTPS.
De hecho, muchos servicios financieros y gubernamentales lo usan en entornos públicos.
- **D. SOAP no permite la autenticación de usuarios en servicios web.**
✗ Falso. SOAP **permite autenticación** mediante varios métodos: UsernameToken, certificados digitales, SAML, etc.

35. ¿Cuál de las siguientes opciones describe mejor el propósito del elemento SOAP Fault?

- A. Proporcionar información adicional sobre el estado de un servicio web.
- B. Indicar que se produjo un error durante la ejecución del servicio.**
- C. Establecer la dirección de destino del mensaje SOAP.
- D. Controlar el flujo de ejecución del servicio.

Justificación:

El elemento **<Fault>** en un mensaje **SOAP** se utiliza para **notificar errores o excepciones** que ocurren durante el **procesamiento de una solicitud** en un servicio web.

Es un elemento **opcional**, pero **cuando aparece dentro del <Body>**, indica que el servidor **no pudo completar la operación solicitada correctamente**.

En otras palabras, **SOAP Fault** actúa como el **mecanismo estándar de manejo de errores** dentro de los mensajes SOAP.

Ubicación del elemento Fault en un mensaje SOAP

El elemento **<Fault>** siempre se encuentra **dentro del <Body>**, reemplazando la respuesta normal cuando ocurre un error.

Ejemplo de mensaje SOAP con un Fault:

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Client</faultcode>
      <faultstring>Solicitud mal formada</faultstring>
      <faultactor>http://example.com/service</faultactor>
      <detail>
        <error>El parámetro 'id' es obligatorio.</error>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

Principales componentes del elemento <Fault>

1. **<faultcode>**
 - Indica el **tipo o categoría del error** (por ejemplo: Client, Server, VersionMismatch, MustUnderstand).
 - Permite identificar si el error ocurrió por una solicitud inválida del cliente o por un problema en el servidor.
2. **<faultstring>**
 - Contiene una **descripción legible** del error (mensaje de texto explicativo).
3. **<faultactor>** (*opcional*)
 - Indica **qué nodo o actor** en la cadena de procesamiento causó el error (útil cuando hay intermediarios SOAP).
4. **<detail>** (*opcional*)
 - Incluye **información técnica más específica** sobre la causa del error (por ejemplo, parámetros inválidos, datos faltantes, etc.).

Ejemplo de tipos de errores SOAP

Código de error	Descripción	Causa
Client	Error causado por una solicitud mal formada o inválida	Datos incorrectos enviados por el cliente
Server	Error interno del servidor	Fallo en la lógica del servicio
VersionMismatch	Versión incompatible del protocolo SOAP	Cliente y servidor usan distintas versiones
MustUnderstand	Un encabezado obligatorio no fue procesado	El servidor no entendió un header crítico

Por qué las demás opciones son incorrectas

- A. Proporcionar información adicional sobre el estado de un servicio web.**
 ✗ Incorrecto. El propósito del <Fault> no es brindar “información adicional”, sino **informar errores concretos** durante la ejecución.
- C. Establecer la dirección de destino del mensaje SOAP.**
 ✗ Incorrecto. La dirección de destino se define en el **envelope o en el encabezado HTTP**, no en el Fault.

- **D. Controlar el flujo de ejecución del servicio.**
✗ Incorrecto. El <Fault> no controla el flujo, solo **reporta** un problema una vez que ocurre.

36. ¿Qué es JAX-WS en el contexto de SOAP en Java?

- A. Java API for XML (JAX) - Web Services.
- B. Java Architecture for XML-based Web Services.
- C. **Java API for XML Web Services.**
- D. Java Advanced XML Web Services.

Justificación:

JAX-WS (Java API for XML Web Services) es la **API estándar de Java** que permite **crear, publicar y consumir servicios web basados en SOAP**.

Forma parte de la especificación **Jakarta EE (antes Java EE)** y proporciona una forma sencilla y eficiente de trabajar con **mensajes SOAP** sin necesidad de manipular directamente el XML.

Significado del acrónimo

Sigla	Significado
JAX	Java API for XML
WS	Web Services

Por tanto, **JAX-WS** significa literalmente:

Java API for XML Web Services

Funciones principales de JAX-WS

1. Permite crear servicios web SOAP en Java

A través de anotaciones como `@WebService` y `@WebMethod`, un desarrollador puede exponer métodos Java como servicios SOAP.

Ejemplo:

```
import javax.jws.WebService;  
import javax.jws.WebMethod;
```

```
@WebService  
public class Calculadora {  
    @WebMethod  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
}
```

Al desplegar esta clase, el contenedor (por ejemplo, GlassFish o Payara) genera automáticamente el **WSDL** correspondiente y expone el servicio.

2. Genera y consume servicios a partir de WSDL

- Con la herramienta `wsimport`, se puede generar código cliente Java a partir de un archivo WSDL existente.
- Con la herramienta `wsgen`, se puede generar el WSDL a partir del código Java del servicio.

3. **Interoperabilidad total**

- JAX-WS sigue los estándares SOAP 1.1 / 1.2, WSDL, y XML Schema, lo que garantiza que los servicios desarrollados en Java puedan comunicarse con clientes de .NET, PHP, Python, C++, etc.

4. **Soporte para seguridad y transporte**

- Compatible con WS-Security, HTTP, HTTPS, y otros mecanismos estándar de mensajería SOAP.

Por qué las demás opciones son incorrectas

• **A. Java API for XML (JAX) - Web Services.**

✗ Incorrecto. “JAX” no significa “Java Architecture for XML”, sino **Java API for XML**. Además, la forma correcta del nombre es **Java API for XML Web Services**, no “(JAX) - Web Services”.

• **B. Java Architecture for XML-based Web Services.**

✗ Incorrecto. No se trata de una arquitectura, sino de una **API** (Application Programming Interface).

• **D. Java Advanced XML Web Services.**

✗ Incorrecto. “Advanced” no forma parte del nombre oficial. Es una confusión común, pero **no corresponde a la definición estandarizada de JAX-WS**.

37. ¿Cuál de las siguientes afirmaciones es verdadera sobre la interoperabilidad en SOAP?

- A. SOAP no es compatible con sistemas no basados en Java.
- B. SOAP permite la interoperabilidad entre diferentes plataformas y lenguajes de programación.**
- C. SOAP solo puede interoperar con servicios web RESTful.
- D. La interoperabilidad es un problema común en servicios web SOAP.

Justificación:

Una de las **principales ventajas de SOAP (Simple Object Access Protocol)** es su capacidad de **interoperar entre diferentes plataformas, sistemas operativos y lenguajes de programación**. Esto significa que **un servicio web SOAP desarrollado en una tecnología puede ser consumido por un cliente escrito en otra**, siempre que ambos respeten los **estándares del protocolo SOAP y WSDL**.

Qué significa interoperabilidad en SOAP

La **interoperabilidad** es la capacidad de dos o más sistemas (independientemente de su tecnología) de **comunicarse y compartir información** de manera efectiva.

SOAP logra esto porque:

1. **Utiliza estándares abiertos y universales:**

- **XML** → para estructurar los datos del mensaje.
- **WSDL (Web Services Description Language)** → para describir el servicio y sus operaciones.
- **HTTP / HTTPS** → como protocolo de transporte común y ampliamente soportado.

2. **No depende de un lenguaje o plataforma específica.**

Por ejemplo:

- Un servicio SOAP desarrollado en **Java (JAX-WS)** puede ser consumido por un cliente **.NET, Python, PHP, o C++** sin problemas.

- Esto es posible porque todos los sistemas entienden el mismo **formato estándar XML** definido por SOAP.
- 3. **Está diseñado con compatibilidad cruzada en mente.**
SOAP fue creado por el **W3C** precisamente para lograr una **comunicación universal** entre **aplicaciones distribuidas**.

Ejemplo de interoperabilidad

Servidor (Java - JAX-WS):

```
@WebService
public class Calculadora {
    @WebMethod
    public int sumar(int a, int b) {
        return a + b;
    }
}
```

Cliente (C# - .NET):

```
CalculadoraClient client = new CalculadoraClient();
int resultado = client.sumar(5, 3);
Console.WriteLine(resultado); // Salida: 8
```

A pesar de usar **lenguajes y entornos diferentes**, ambos sistemas pueden comunicarse gracias al **contrato WSDL** y al protocolo **SOAP**.

Por qué las demás opciones son incorrectas

- **A. SOAP no es compatible con sistemas no basados en Java.**
✗ Falso. SOAP **no depende de Java**; puede usarse en .NET, PHP, Python, C++, Ruby, etc.
- **C. SOAP solo puede interoperar con servicios web RESTful.**
✗ Incorrecto. SOAP y REST son **estilos diferentes de servicios web**. SOAP **no necesita ni depende de REST** para funcionar.
- **D. La interoperabilidad es un problema común en servicios web SOAP.**
✗ Falso. Todo lo contrario: SOAP fue diseñado **precisamente para resolver los problemas de interoperabilidad** entre plataformas heterogéneas.

38. ¿Cuál de las siguientes opciones describe mejor la relación entre SOAP y WSDL?

- A. SOAP es una especificación de WSDL.
- B. WSDL es una especificación de SOAP.
- C. SOAP utiliza WSDL para definir la estructura y el formato de los mensajes.**
- D. SOAP y WSDL son términos intercambiables y se refieren a lo mismo.

Justificación:

SOAP y **WSDL** son dos componentes fundamentales en el desarrollo de **servicios web basados en XML**, pero **cumplen funciones diferentes y complementarias**.

Relación entre SOAP y WSDL

1. **SOAP (Simple Object Access Protocol)**
Es un **protocolo de mensajería** basado en **XML** que define **cómo se envían y reciben mensajes** entre aplicaciones distribuidas.
2. **WSDL (Web Services Description Language)**
Es un **lenguaje de descripción** (también basado en XML) que **define la interfaz pública del servicio web**, es decir:

- Qué operaciones ofrece el servicio.
- Qué parámetros recibe y qué devuelve.
- Qué tipo de datos se utilizan.
- Dónde se encuentra el servicio (su **URL**).
- Qué protocolo se usa para comunicarse (por ejemplo, **SOAP sobre HTTP**).

3. **Relación directa:**

SOAP usa WSDL como contrato para que tanto el cliente como el servidor **entiendan exactamente cómo comunicarse**.

En otras palabras:

El archivo **WSDL describe cómo deben ser los mensajes SOAP**.

Ejemplo simple

Archivo WSDL (extracto):

```
<definitions name="CalculadoraService"
  targetNamespace="http://ejemplo.com/calculadora"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://ejemplo.com/calculadora"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <message name="sumarRequest">
    <part name="a" type="xsd:int"/>
    <part name="b" type="xsd:int"/>
  </message>

  <message name="sumarResponse">
    <part name="resultado" type="xsd:int"/>
  </message>

  <portType name="CalculadoraPortType">
    <operation name="sumar">
      <input message="tns:sumarRequest"/>
      <output message="tns:sumarResponse"/>
    </operation>
  </portType>
</definitions>
```

Este archivo **describe cómo debe estructurarse un mensaje SOAP** para invocar la operación `sumar`.

Por qué las demás opciones son incorrectas

- **A. SOAP es una especificación de WSDL.**
✗ Incorrecto. SOAP **no depende de WSDL** para existir; es un protocolo independiente.
- **B. WSDL es una especificación de SOAP.**
✗ Falso. WSDL es un **lenguaje de descripción**, no una especificación de SOAP; también puede describir servicios REST.
- **D. SOAP y WSDL son términos intercambiables y se refieren a lo mismo.**
✗ Totalmente incorrecto.
SOAP es un **protocolo de comunicación**, mientras que WSDL es un **documento que describe** cómo se usa ese protocolo.

39. ¿Cuál de las siguientes opciones describe mejor el uso de SOAP en comparación con REST en términos de complejidad?

- A. SOAP es más simple y liviano que REST.
- B. SOAP es más complejo y pesado que REST.
- C. SOAP y REST tienen la misma complejidad.
- D. La complejidad depende del lenguaje de programación utilizado.

Justificación:

SOAP (Simple Object Access Protocol) y **REST (Representational State Transfer)** son dos enfoques distintos para la creación de **servicios web**, pero difieren significativamente en su **complejidad, estructura y flexibilidad**.

Comparación entre SOAP y REST

Característica	SOAP	REST
Tipo de tecnología	Protocolo de mensajería basado en XML.	Estilo arquitectónico basado en HTTP.
Formato de datos	Solo XML.	JSON, XML, YAML, u otros.
Estructura del mensaje	Muy estructurada y con encabezados (Envelope, Header, Body, Fault).	Ligera y flexible, generalmente usa JSON.
Uso de red	Puede usar HTTP, SMTP, u otros protocolos.	Normalmente usa HTTP.
Seguridad y transacciones	Incorpora estándares avanzados (WS-Security, WS-AtomicTransaction).	Requiere mecanismos externos (como HTTPS y OAuth).
Complejidad	Alta: requiere WSDL, XML, y especificaciones adicionales.	Baja: más sencillo de implementar y entender.

Explicación

- **SOAP es más complejo y pesado** porque:
 1. Usa **mensajes XML extensos**, que deben cumplir con un formato rígido (Envelope, Header, Body, Fault).
 2. Depende de un **WSDL (Web Services Description Language)** para describir sus operaciones.
 3. Maneja una gran cantidad de **estándares complementarios** (WS-Security, WS-Policy, WS-ReliableMessaging, etc.).
 4. Su configuración y validación requieren **más procesamiento** tanto en el servidor como en el cliente.
- En cambio, **REST**:
 - Se basa en **HTTP estándar** (GET, POST, PUT, DELETE).
 - Usa **JSON** como formato principal, mucho más ligero.
 - Es **más fácil de implementar y mantener**.

Por qué las demás opciones son incorrectas

- **A. SOAP es más simple y liviano que REST.**
✗ Incorrecto. SOAP es más **pesado y complejo** debido al formato XML y a la rigidez del protocolo.
- **C. SOAP y REST tienen la misma complejidad.**
✗ Falso. REST fue diseñado precisamente para **reducir la complejidad** que tenía SOAP.

- **D. La complejidad depende del lenguaje de programación utilizado.**
✗ Parcialmente cierto, pero no relevante aquí.
La **complejidad proviene del protocolo**, no del lenguaje que se use para implementarlo.

40. ¿Cuál de las siguientes opciones describe mejor el propósito del elemento SOAP Action?

- A. Proporcionar información adicional en el cuerpo del mensaje SOAP.
- B. Identificar el tipo de acción o método que se ejecutará en el servicio web.**
- C. Definir las reglas de validación para el mensaje SOAP.
- D. Establecer la dirección de destino del mensaje SOAP.

Justificación:

El **SOAP Action** es un **encabezado HTTP** que se utiliza junto con los mensajes SOAP para **indicar al servicio web qué operación o método se desea invocar**.

Es especialmente útil cuando un **endpoint SOAP expone múltiples operaciones**, permitiendo que el servidor identifique **precisamente qué acción debe ejecutar**.

Cómo funciona SOAP Action

1. **Se envía en la cabecera HTTP del mensaje SOAP:**
POST /CalculadoraService HTTP/1.1
Host: example.com
Content-Type: text/xml; charset=utf-8
SOAPAction: "http://example.com/calculadora/sumar"
Content-Length: 456
2. **Corresponde a la operación definida en el WSDL:**
 - WSDL define un **binding** que asocia cada operación con un **SOAPAction** único.
 - Cuando el servidor recibe el mensaje, utiliza el valor de SOAPAction para **dirigir el mensaje a la operación correcta**.
3. **Permite distinguir entre múltiples métodos en el mismo endpoint:**
 - Por ejemplo, un servicio SOAP de **Calculadora** podría tener operaciones sumar, restar, multiplicar.
 - Cada operación tendría un **SOAPAction distinto** que el servidor utiliza para ejecutar la función adecuada.

Por qué las demás opciones son incorrectas

- **A. Proporcionar información adicional en el cuerpo del mensaje SOAP.**
✗ Incorrecto. El **cuerpo (Body)** contiene los datos principales de la operación, no la identificación de la acción.
- **C. Definir las reglas de validación para el mensaje SOAP.**
✗ Incorrecto. La validación de SOAP se hace mediante **XML Schema** o WSDL, no a través de SOAPAction.
- **D. Establecer la dirección de destino del mensaje SOAP.**
✗ Incorrecto. La **dirección del mensaje** se define en el **endpoint** dentro del WSDL o en la URL de la solicitud HTTP, no en SOAPAction.

41. ¿Cuál de las siguientes opciones describe mejor el enfoque de "contrato primero" en el desarrollo de servicios web SOAP?

- A. Se implementa el servicio web y luego se define su contrato.
- B. Se define el contrato del servicio web antes de su implementación.**
- C. No es aplicable en el desarrollo de servicios web SOAP.
- D. El enfoque de "contrato primero" no se utiliza en SOAP.

Justificación:

El enfoque “**contrato primero**” (**contract-first**) en SOAP implica que **el desarrollo del servicio web inicia con la definición del contrato**, que generalmente se realiza mediante un **archivo WSDL (Web Services Description Language)**.

El contrato describe **qué operaciones ofrece el servicio, los tipos de datos que intercambia y cómo deben ser estructurados los mensajes SOAP**, sirviendo como guía para la implementación posterior.

Cómo funciona el enfoque “contrato primero”

1. **Definir el contrato (WSDL)**
 - Se especifican las **operaciones**, los **mensajes de entrada y salida**, los **tipos de datos** y el **endpoint del servicio**.
 - Ejemplo de operación en WSDL:

```
<operation name="sumar">
  <input message="tns:sumarRequest"/>
  <output message="tns:sumarResponse"/>
</operation>
```
2. **Generar el código a partir del WSDL**
 - Herramientas como wsimport (Java) permiten generar automáticamente las **clases servidor y cliente** que cumplen con el contrato definido.
3. **Implementar la lógica del servicio**
 - El desarrollador implementa los métodos Java asegurándose de que **respeten el contrato definido en WSDL**.

Ventajas del enfoque “contrato primero”

- **Interoperabilidad:** Clientes en cualquier lenguaje pueden consumir el servicio usando el mismo WSDL.
- **Claridad:** El contrato actúa como un acuerdo formal entre el cliente y el servidor.
- **Validación fácil:** Los mensajes SOAP pueden ser validados contra el esquema definido en el WSDL.
- **Menor acoplamiento:** El cliente y el servidor dependen únicamente del contrato, no de la implementación interna.

Por qué las demás opciones son incorrectas

- **A. Se implementa el servicio web y luego se define su contrato.**
✗ Este es el enfoque “**contract-last**”, no “contrato primero”.
- **C. No es aplicable en el desarrollo de servicios web SOAP.**
✗ Falso. “Contrato primero” es un enfoque **muy común en SOAP**.
- **D. El enfoque de "contrato primero" no se utiliza en SOAP.**
✗ Incorrecto. SOAP fue diseñado para trabajar con **contratos WSDL**, y “contrato primero” es ampliamente recomendado.

42. ¿Cuál de las siguientes opciones describe mejor la función del elemento SOAP Body?

- A. Contiene la información principal del mensaje SOAP.**
- B. Define la dirección de destino del mensaje SOAP.
- C. Proporciona información adicional sobre la solicitud o respuesta SOAP.
- D. Establece las reglas de validación para el mensaje SOAP.

Justificación:

El elemento **<Body>** en un mensaje SOAP es **obligatorio** y contiene la **información principal** que se está transmitiendo entre el cliente y el servidor. Esto incluye:

- **Solicitudes:** datos que el cliente envía para invocar una operación en el servicio web.
- **Respuestas:** datos que el servidor devuelve después de procesar la solicitud.
- **Mensajes de error:** cuando ocurre un problema, el **<Body>** puede contener un **<Fault>** para reportarlo.

Ubicación en el mensaje SOAP

El **<Body>** se encuentra **dentro del <Envelope>** y **después del <Header>** (si existe). Su estructura básica es:

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">  
  <soap:Header>  
    <!-- Información opcional, como autenticación -->  
  </soap:Header>  
  
  <soap:Body>  
    <m:getProducto xmlns:m="http://example.com/tienda">  
      <m:id>25</m:id>  
    </m:getProducto>  
  </soap:Body>  
</soap:Envelope>
```

- **<Body>:** contiene la **operación y los datos asociados** (ej. `getProducto` y su parámetro `id`).
- **<Header>:** contiene información **adicional u opcional**, como tokens de seguridad, transacciones o metadatos.

Por qué las demás opciones son incorrectas

- **B. Define la dirección de destino del mensaje SOAP.**
✗ Incorrecto. La **dirección del servicio** se define en el **endpoint del WSDL** o la URL del HTTP request, no en el **<Body>**.
- **C. Proporciona información adicional sobre la solicitud o respuesta SOAP.**
✗ Incorrecto. La información adicional opcional se coloca en el **Header**, no en el **<Body>**.
- **D. Establece las reglas de validación para el mensaje SOAP.**
✗ Falso. La validación de mensajes SOAP se realiza mediante **XML Schema** o definiciones del WSDL, no en el **<Body>**.

43. ¿Cuál de las siguientes afirmaciones es verdadera sobre SOAP y seguridad?

- A. SOAP no es compatible con mecanismos de seguridad, como la encriptación y la autenticación.
- B. SOAP tiene un alto nivel de seguridad incorporado mediante la firma y el cifrado de mensajes.
- C. SOAP solo se puede utilizar en redes privadas y no es seguro para su uso en Internet.
- D. SOAP no permite la autenticación de usuarios en servicios web.

Justificación:

SOAP es un protocolo diseñado para **intercambiar mensajes estructurados entre aplicaciones distribuidas**. Una de sus ventajas clave es que **puede implementar seguridad a nivel de mensaje**, independientemente del transporte, gracias a la especificación **WS-Security (Web Services Security)**.

Características de seguridad de SOAP

1. **Firma digital**
 - Permite verificar la **integridad** y **autenticidad** del mensaje.
 - Garantiza que el contenido no ha sido modificado y que proviene de un remitente confiable.
2. **Cifrado de mensajes**
 - Protege la **confidencialidad** de la información dentro del mensaje SOAP.
 - Solo los destinatarios autorizados pueden descifrar el contenido.
3. **Autenticación de usuarios**
 - SOAP puede incluir tokens de seguridad como **UsernameToken**, **BinarySecurityToken** o **SAML Token** en el <Header> del mensaje.
 - Esto permite que los servicios web autenticen usuarios antes de procesar solicitudes.
4. **Seguridad independiente del transporte**
 - A diferencia de HTTPS, que protege solo el canal de comunicación, **WS-Security protege el mensaje completo**, incluso si pasa por múltiples intermediarios.

Ejemplo de Header SOAP con seguridad

```
<soap:Header>
  <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/">
    <wsse:UsernameToken>
      <wsse:Username>usuario123</wsse:Username>
      <wsse:Password>claveSegura</wsse:Password>
    </wsse:UsernameToken>
  </wsse:Security>
</soap:Header>
```

Por qué las demás opciones son incorrectas

- **A. SOAP no es compatible con mecanismos de seguridad, como la encriptación y la autenticación.**
✗ Falso. SOAP **sí soporta** cifrado, firma y autenticación mediante WS-Security.
- **C. SOAP solo se puede utilizar en redes privadas y no es seguro para su uso en Internet.**
✗ Incorrecto. SOAP es seguro para Internet cuando se combina con WS-Security o HTTPS.
- **D. SOAP no permite la autenticación de usuarios en servicios web.**
✗ Falso. SOAP permite **autenticación de usuarios** mediante tokens y certificados en el Header.

44. ¿Cuál de las siguientes opciones describe mejor el propósito del elemento SOAP Header?

- A. Contiene la información principal del mensaje SOAP.
- B. Es utilizado para autenticar la solicitud SOAP.
- C. Proporciona información adicional y opcional sobre la solicitud o respuesta SOAP.**
- D. Define las reglas de validación para el mensaje SOAP.

Justificación:

El elemento <Header> en SOAP es **opcional** y se utiliza para incluir **información adicional que acompaña al mensaje principal** (contenido del <Body>). Esta información puede ser utilizada por **intermediarios o el servicio receptor**, y puede incluir:

- **Autenticación y seguridad** (por ejemplo, tokens de WS-Security).
- **Datos de transacción o sesión.**
- **Metadatos o instrucciones adicionales** para el procesamiento del mensaje.

Ejemplo de un Header SOAP

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/">
      <wsse:UsernameToken>
        <wsse:Username>usuario123</wsse:Username>
        <wsse:Password>claveSegura</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </soap:Header>

  <soap:Body>
    <m:getProducto xmlns:m="http://example.com/tienda">
      <m:id>25</m:id>
    </m:getProducto>
  </soap:Body>
</soap:Envelope>
```

- <Header>: contiene información **opcional**, como autenticación.
- <Body>: contiene la **información principal** del mensaje SOAP.

Por qué las demás opciones son incorrectas

- **A. Contiene la información principal del mensaje SOAP.**
✗ Incorrecto. La información principal siempre se coloca en el <Body>.
- **B. Es utilizado para autenticar la solicitud SOAP.**
✗ Parcialmente cierto, pero el Header no se limita a autenticación; puede incluir cualquier información adicional opcional.
- **D. Define las reglas de validación para el mensaje SOAP.**
✗ Falso. La validación se hace mediante **XML Schema** o WSDL, no mediante el Header.

45. ¿Cuál de las siguientes opciones describe mejor la relación entre SOAP y XML?

- A. SOAP es un formato de mensaje basado en XML.**
- B. XML es un formato de mensaje basado en SOAP.
- C. SOAP y XML son protocolos de comunicación diferentes pero complementarios.
- D. SOAP y XML son términos intercambiables y se refieren a lo mismo.

Justificación:

SOAP (Simple Object Access Protocol) es un **protocolo de mensajería** que se utiliza para **intercambiar información estructurada entre aplicaciones distribuidas**, y siempre utiliza **XML** como su **formato de mensaje subyacente**.

- Esto significa que todos los **mensajes SOAP** están escritos en **XML**, con elementos como <Envelope>, <Header> y <Body>.
- XML proporciona **estructura, legibilidad y estandarización**, permitiendo que SOAP sea **independiente de la plataforma y del lenguaje de programación**.

Ejemplo de mensaje SOAP en XML

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <!-- Información opcional -->
  </soap:Header>
```

```
<soap:Body>
  <m:getProducto xmlns:m="http://example.com/tienda">
    <m:id>25</m:id>
  </m:getProducto>
</soap:Body>
</soap:Envelope>
```

- <Envelope> y <Body> son **elementos SOAP**, escritos en XML.
- Sin XML, **SOAP no podría estructurar ni transportar los datos de manera estándar.**

Por qué las demás opciones son incorrectas

- **B. XML es un formato de mensaje basado en SOAP.**
✗ Incorrecto. XML **existe independientemente de SOAP** y se utiliza en muchos otros contextos, como documentos, configuraciones y otros protocolos.
- **C. SOAP y XML son protocolos de comunicación diferentes pero complementarios.**
✗ Falso. SOAP es un **protocolo**, XML es un **formato de datos**, no un protocolo de comunicación.
- **D. SOAP y XML son términos intercambiables y se refieren a lo mismo.**
✗ Incorrecto. SOAP **usa XML**, pero no son equivalentes ni intercambiables.

46. ¿Cuál de las siguientes opciones describe mejor el enfoque de "contrato primero" en el desarrollo de servicios web SOAP?

- A. Se implementa el servicio web y luego se define su contrato.
- B. Se define el contrato del servicio web antes de su implementación.**
- C. No es aplicable en el desarrollo de servicios web SOAP.
- D. El enfoque de "contrato primero" no se utiliza en SOAP.

Justificación:

El enfoque **"contrato primero" (contract-first)** en SOAP consiste en **definir primero el contrato del servicio web**, generalmente mediante un **archivo WSDL**, antes de escribir cualquier código del servicio.

El contrato describe:

- Qué **operaciones** ofrece el servicio.
- Qué **parámetros y tipos de datos** se utilizan.
- Cómo se estructuran los **mensajes SOAP**.
- La **dirección del endpoint** del servicio.

Una vez definido el WSDL, los desarrolladores implementan la lógica del servicio asegurándose de que **cumpla exactamente con el contrato especificado.**

Ventajas del enfoque "contrato primero"

1. **Interoperabilidad:** Cualquier cliente que entienda WSDL puede consumir el servicio, sin importar el lenguaje o la plataforma.
2. **Claridad:** El contrato actúa como un acuerdo formal entre cliente y servidor.
3. **Validación de mensajes:** Los mensajes SOAP pueden ser validados automáticamente contra el WSDL.
4. **Menor acoplamiento:** Los cambios en la implementación no afectan el contrato mientras se cumpla WSDL.

Por qué las demás opciones son incorrectas

- **A. Se implementa el servicio web y luego se define su contrato.**
✗ Esto corresponde al enfoque **"contract-last"**, no "contrato primero".

- **C. No es aplicable en el desarrollo de servicios web SOAP.**
✗ Falso. El enfoque “contrato primero” es ampliamente utilizado y recomendado en SOAP.
- **D. El enfoque de "contrato primero" no se utiliza en SOAP.**
✗ Incorrecto. SOAP está diseñado para trabajar con WSDL y se beneficia del enfoque “contrato primero”.

47. ¿Cuál de las siguientes opciones describe mejor la función del elemento SOAP Envelope?

- A. Define la estructura y el formato de los mensajes SOAP.
- B. Proporciona información adicional en el cuerpo del mensaje SOAP.
- C. Envuelve el mensaje SOAP completo y lo identifica como tal.**
- D. Contiene la información principal del mensaje SOAP.

Justificación:

El elemento **<Envelope>** es el **elemento raíz de todos los mensajes SOAP** y cumple las siguientes funciones:

1. **Identificar el mensaje como un mensaje SOAP:**
 - Todo mensaje SOAP debe comenzar con **<Envelope>** para que el receptor lo reconozca como un mensaje válido.
2. **Encapsular todos los elementos del mensaje:**
 - Dentro del **<Envelope>** se incluyen:
 - **<Header>** (opcional): información adicional, como seguridad, transacciones o metadatos.
 - **<Body>** (obligatorio): contiene la información principal del mensaje o un **<Fault>** en caso de error.

Estructura básica de un mensaje SOAP

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <!-- Información opcional como autenticación -->
  </soap:Header>
  <soap:Body>
    <!-- Información principal o Fault -->
    <m:getProducto xmlns:m="http://example.com/tienda">
      <m:id>25</m:id>
    </m:getProducto>
  </soap:Body>
</soap:Envelope>
```

- **<Envelope>**: **envuelve todo el mensaje SOAP** y lo identifica claramente.
- **<Header>**: opcional, para información adicional.
- **<Body>**: obligatorio, para datos de la operación o errores.

Por qué las demás opciones son incorrectas

- **A. Define la estructura y el formato de los mensajes SOAP.**
✗ Parcialmente cierto, pero la definición de la estructura exacta se encuentra en **la especificación SOAP**; **<Envelope>** solo **encapsula** el mensaje.
- **B. Proporciona información adicional en el cuerpo del mensaje SOAP.**
✗ Incorrecto. El **<Body>** es el que contiene la información principal, no el **<Envelope>**.
- **D. Contiene la información principal del mensaje SOAP.**
✗ Incorrecto. El **<Body>** contiene la información principal; **<Envelope>** solo **la envuelve**.

48. ¿Cuál de las siguientes afirmaciones es verdadera sobre SOAP y REST?

- A. SOAP es más adecuado para aplicaciones web simples y ligeras, mientras que REST es más adecuado para aplicaciones empresariales complejas.
- B. SOAP es más simple y liviano que REST.
- C. SOAP y REST son protocolos de comunicación diferentes pero complementarios.
- D. SOAP y REST son arquitecturas de servicios web diferentes y tienen enfoques distintos.**

Justificación:

SOAP y REST son dos enfoques distintos para crear **servicios web**, con diferencias clave en su **arquitectura, complejidad y estilo de comunicación**:

Comparación SOAP vs REST

Característica	SOAP	REST
Naturaleza	Protocolo de mensajería estandarizado.	Estilo arquitectónico basado en HTTP.
Formato de mensaje	XML (estricto y pesado).	JSON, XML u otros formatos ligeros.
Transporte	Puede usar HTTP, SMTP, JMS, etc.	Normalmente HTTP(S).
Seguridad	WS-Security (firma, cifrado).	HTTPS, OAuth, JWT.
Complejidad	Alta, más rígido y estructurado.	Bajo, más flexible y ligero.
Uso típico	Aplicaciones empresariales complejas, integraciones entre sistemas.	Aplicaciones web y móviles ligeras, APIs públicas.

Conclusión clave

- SOAP y REST **no son equivalentes ni intercambiables**.
- Tienen **enfoques distintos**: SOAP se centra en **protocolos y estándares**, REST se centra en **recursos y operaciones HTTP**.
- Cada uno es **adecuado según el tipo de aplicación y los requisitos**.

Por qué las demás opciones son incorrectas

- **A. SOAP es más adecuado para aplicaciones web simples y ligeras, mientras que REST es más adecuado para aplicaciones empresariales complejas.**
✗ Falso. Es justamente al revés: SOAP se usa en aplicaciones empresariales complejas, REST en aplicaciones ligeras.
- **B. SOAP es más simple y liviano que REST.**
✗ Incorrecto. SOAP es más **complejo y pesado** que REST.
- **C. SOAP y REST son protocolos de comunicación diferentes pero complementarios.**
✗ Parcialmente cierto, pero REST no es un protocolo; es un **estilo arquitectónico**.

SOAP y REST son **enfoques diferentes para servicios web**, cada uno con sus ventajas y desventajas, y no se usan de la misma manera ni con los mismos objetivos.

49. ¿Cuál de las siguientes opciones describe mejor el propósito del elemento SOAP Fault?

- A. Proporcionar información adicional sobre el estado de un servicio web.
- B. Indicar que se produjo un error durante la ejecución del servicio.**
- C. Establecer la dirección de destino del mensaje SOAP.
- D. Controlar el flujo de ejecución del servicio.

Justificación:

El elemento **<Fault>** en SOAP se utiliza **exclusivamente para reportar errores** que ocurren durante el procesamiento de un mensaje SOAP. Se encuentra **dentro del <Body>** y proporciona información estructurada sobre el fallo.

Estructura típica de un SOAP Fault

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Client</faultcode>
      <faultstring>Error: Parámetro inválido</faultstring>
      <detail>
        <m:invalidParameter xmlns:m="http://example.com/errors">
          El parámetro 'id' debe ser mayor que 0.
        </m:invalidParameter>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

- <faultcode>: tipo de error (Client, Server, etc.).
- <faultstring>: mensaje descriptivo del error.
- <detail>: información adicional específica sobre el fallo.

Por qué las demás opciones son incorrectas

- **A. Proporcionar información adicional sobre el estado de un servicio web.**
✗ Incorrecto. <Fault> indica **un error**, no simplemente información adicional.
- **C. Establecer la dirección de destino del mensaje SOAP.**
✗ Falso. La dirección del mensaje se define en el **endpoint**, no en <Fault>.
- **D. Controlar el flujo de ejecución del servicio.**
✗ Incorrecto. <Fault> **informa errores**, pero no controla el flujo de ejecución.

El elemento **<Fault>** se utiliza para **informar de errores en la ejecución del servicio**, proporcionando detalles estructurados para que el cliente pueda manejar la situación correctamente.

50. ¿Cuál de las siguientes opciones describe mejor el propósito del elemento SOAP Action?

- A. Proporcionar información adicional en el cuerpo del mensaje SOAP.
- B. Identificar el tipo de acción o método que se ejecutará en el servicio web.**
- C. Definir las reglas de validación para el mensaje SOAP.
- D. Establecer la dirección de destino del mensaje SOAP.

Justificación:

El **SOAP Action** es un **encabezado HTTP** que se utiliza junto con los mensajes SOAP para **indicar al servicio web qué operación o método debe invocarse**. Es especialmente útil cuando un **endpoint SOAP expone múltiples operaciones**, permitiendo al servidor **identificar la acción correcta**.

Cómo funciona SOAP Action

1. **Se envía en la cabecera HTTP del mensaje SOAP:**
POST /CalculadoraService HTTP/1.1
Host: example.com
Content-Type: text/xml; charset=utf-8
SOAPAction: "http://example.com/calculadora/sumar"
Content-Length: 456
2. **Corresponde a la operación definida en el WSDL:**
 - WSDL asigna un valor único de SOAPAction a cada operación.
 - El servidor utiliza este valor para **dirigir el mensaje a la operación correcta**.
3. **Permite diferenciar múltiples métodos en el mismo endpoint:**
 - Ejemplo: operaciones sumar, restar, multiplicar en un mismo servicio.
 - Cada operación tiene un **SOAPAction distinto**.

Por qué las demás opciones son incorrectas

- **A. Proporcionar información adicional en el cuerpo del mensaje SOAP.**
✗ Incorrecto. La información principal se envía en el <Body>, no en SOAPAction.
- **C. Definir las reglas de validación para el mensaje SOAP.**
✗ Falso. La validación se realiza mediante **XML Schema** o WSDL.
- **D. Establecer la dirección de destino del mensaje SOAP.**
✗ Incorrecto. La dirección se define en el **endpoint del WSDL** o en la URL de la solicitud HTTP.

El **propósito de SOAPAction** es **identificar la acción o método específico que se ejecutará** en el servicio web, asegurando que un mismo endpoint pueda manejar múltiples operaciones de manera correcta.

51. ¿Cuál de las siguientes afirmaciones es verdadera sobre SOAP y seguridad?

- A. SOAP no es compatible con mecanismos de seguridad, como la encriptación y la autenticación.
- B. **SOAP tiene un alto nivel de seguridad incorporado mediante la firma y el cifrado de mensajes.**
- C. SOAP solo se puede utilizar en redes privadas y no es seguro para su uso en Internet.
- D. SOAP no permite la autenticación de usuarios en servicios web.

Justificación:

SOAP es un protocolo diseñado para **intercambiar mensajes estructurados entre aplicaciones distribuidas**. Su seguridad se implementa mediante la especificación **WS-Security**, lo que permite:

1. **Firma digital**
 - Garantiza la **integridad y autenticidad** del mensaje.
 - Asegura que el contenido no ha sido modificado y que proviene de un remitente confiable.
2. **Cifrado de mensajes**
 - Protege la **confidencialidad** de los datos.
 - Solo los destinatarios autorizados pueden descifrar el contenido.
3. **Autenticación de usuarios**
 - SOAP permite incluir **tokens de seguridad** como UsernameToken, BinarySecurityToken o SAML Token en el <Header>.
 - Esto asegura que solo usuarios autorizados puedan consumir el servicio.

4. **Seguridad independiente del transporte**

- WS-Security protege el mensaje completo, incluso si pasa por múltiples intermediarios, a diferencia de HTTPS que protege solo el canal.

Por qué las demás opciones son incorrectas

- **A. SOAP no es compatible con mecanismos de seguridad, como la encriptación y la autenticación.**
✗ Falso. SOAP soporta **firma, cifrado y autenticación** mediante WS-Security.
- **C. SOAP solo se puede utilizar en redes privadas y no es seguro para su uso en Internet.**
✗ Incorrecto. SOAP es seguro para Internet si se combina con WS-Security o HTTPS.
- **D. SOAP no permite la autenticación de usuarios en servicios web.**
✗ Falso. SOAP **permite autenticación** mediante tokens o certificados en el <Header>.

SOAP proporciona **mecanismos avanzados de seguridad** para proteger la integridad, confidencialidad y autenticación de mensajes, haciéndolo confiable incluso en entornos abiertos.

52. **¿Qué es JAX-WS en el contexto de SOAP en Java?**

- A. Java API for XML (JAX) - Web Services.
- B. Java Architecture for XML-based Web Services.
- C. Java API for XML Web Services.**
- D. Java Advanced XML Web Services.

Justificación:

JAX-WS es la **API estándar de Java para desarrollar y consumir servicios web SOAP**. Su propósito principal es permitir a los desarrolladores:

- **Crear servicios web SOAP** en Java de manera sencilla.
- **Consumir servicios web SOAP** desde aplicaciones Java.
- **Automatizar el mapeo entre objetos Java y mensajes XML/SOAP**, mediante anotaciones (@WebService, @WebMethod, etc.).

⚙️ **Características clave de JAX-WS**

1. **Basado en anotaciones**

Permite marcar clases y métodos Java para exponerlos como operaciones SOAP:

```
import jakarta.jws.WebService;  
import jakarta.jws.WebMethod;
```

```
@WebService  
public class CalculadoraService {  
    @WebMethod  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
}
```

2. **Generación de WSDL automática**

- JAX-WS puede generar automáticamente un **archivo WSDL** que describe el servicio.
- Esto facilita el consumo por clientes SOAP de distintos lenguajes.

3. **Compatibilidad con WS- standards***

- Permite implementar seguridad (WS-Security), transacciones y otros estándares de servicios web SOAP.

⊗ Por qué las demás opciones son incorrectas

- **A. Java API for XML (JAX) - Web Services.**
✗ Incorrecto. El nombre correcto es **Java API for XML Web Services**; no es solo JAX.
- **B. Java Architecture for XML-based Web Services.**
✗ Incorrecto. Esto es una confusión con el nombre; JAX-WS es una **API**, no una arquitectura.
- **D. Java Advanced XML Web Services.**
✗ Incorrecto. No existe una especificación oficial con este nombre.

JAX-WS es la **API oficial de Java** para desarrollar servicios web **SOAP**, facilitando la creación, consumo y estandarización de mensajes XML entre aplicaciones distribuidas.

53. ¿Cuál de las siguientes afirmaciones es verdadera sobre la interoperabilidad en SOAP?

- A. SOAP no es compatible con sistemas no basados en Java.
- B. SOAP permite la interoperabilidad entre diferentes plataformas y lenguajes de programación.**
- C. SOAP solo puede interoperar con servicios web RESTful.
- D. La interoperabilidad es un problema común en servicios web SOAP.

Justificación:

SOAP fue diseñado para ser **plataforma y lenguaje independiente**, lo que significa que **un cliente en cualquier lenguaje puede consumir un servicio SOAP implementado en otro lenguaje** siempre que siga el **estándar SOAP** y el contrato WSDL.

Cómo se logra la interoperabilidad

1. **Uso de XML**
 - SOAP utiliza XML como formato de mensaje, lo que permite que cualquier sistema que pueda procesar XML entienda los mensajes.
2. **Definición mediante WSDL**
 - El WSDL describe el **contrato del servicio**, incluyendo operaciones, tipos de datos y formato de los mensajes.
 - Esto permite que clientes en diferentes lenguajes generen código automáticamente para consumir el servicio.
3. **Estándares adicionales (WS-*)**
 - SOAP soporta estándares como WS-Security, WS-Addressing y WS-ReliableMessaging, manteniendo la interoperabilidad incluso con características avanzadas.

Por qué las demás opciones son incorrectas

- **A. SOAP no es compatible con sistemas no basados en Java**
✗ Falso. SOAP funciona en **cualquier lenguaje y plataforma** que implemente el estándar SOAP.
- **C. SOAP solo puede interoperar con servicios web RESTful.**
✗ Incorrecto. REST es un enfoque diferente; SOAP no depende de REST para interoperar.
- **D. La interoperabilidad es un problema común en servicios web SOAP.**
✗ Falso. La interoperabilidad es **una de las principales ventajas de SOAP**, no un problema.

SOAP permite que **distintas plataformas y lenguajes de programación se comuniquen de manera confiable** mediante el uso de XML y WSDL, garantizando interoperabilidad entre sistemas heterogéneos.

54. ¿Cuál de las siguientes opciones describe mejor el uso de SOAP en comparación con REST en términos de complejidad?

- A. SOAP es más simple y liviano que REST.
- B. SOAP es más complejo y pesado que REST.
- C. SOAP y REST tienen la misma complejidad.
- D. La complejidad depende del lenguaje de programación utilizado.

Justificación:

SOAP y REST son dos enfoques distintos para servicios web, y se diferencian en **complejidad y peso**:

Comparación en términos de complejidad

Característica	SOAP	REST
Formato de mensaje	XML estricto y detallado.	JSON, XML u otros formatos ligeros.
Estándares y protocolos	WS-Security, WS-ReliableMessaging, WS-Addressing.	HTTP y sus métodos (GET, POST, PUT, DELETE).
Transporte	Puede usar HTTP, SMTP, JMS, etc.	Principalmente HTTP.
Implementación	Más rígida y estructurada, requiere definición de WSDL y generación de clases.	Más flexible y simple, no requiere WSDL.
Uso típico	Integraciones empresariales complejas, transacciones seguras.	Aplicaciones web y móviles ligeras.

Conclusión clave

- SOAP requiere **más configuración, más protocolos y estándares adicionales**, y sus mensajes son **más pesados** debido al uso obligatorio de XML.
- REST es más **ligero y fácil de implementar**, utilizando directamente HTTP y formatos simples como JSON.

Por qué las demás opciones son incorrectas

- **A. SOAP es más simple y liviano que REST.**
✗ Falso, es justamente lo contrario.
- **C. SOAP y REST tienen la misma complejidad.**
✗ Incorrecto, SOAP es más pesado y estructurado.
- **D. La complejidad depende del lenguaje de programación utilizado.**
✗ La complejidad depende del **protocolo y estándar**, no del lenguaje.

SOAP es **más complejo y pesado** que REST, especialmente en entornos empresariales que requieren transacciones, seguridad y estándares estrictos.

55. ¿Cuál de las siguientes opciones describe mejor el enfoque de "contrato primero" en el desarrollo de servicios web SOAP?

- A. Se implementa el servicio web y luego se define su contrato.
- B. Se define el contrato del servicio web antes de su implementación.
- C. No es aplicable en el desarrollo de servicios web SOAP.
- D. El enfoque de "contrato primero" no se utiliza en SOAP.

Justificación:

El enfoque **"contrato primero" (contract-first)** en SOAP consiste en **definir primero el contrato del servicio web**, generalmente mediante un **archivo WSDL**, antes de desarrollar la lógica del servicio.

El contrato describe:

- Las **operaciones disponibles** en el servicio.
- Los **tipos de datos y parámetros** que se utilizan.
- La **estructura de los mensajes SOAP**.
- La **URL o endpoint** donde se expondrá el servicio.

Una vez definido el WSDL, los desarrolladores implementan la lógica del servicio asegurándose de **cumplir exactamente con el contrato especificado**, lo que garantiza interoperabilidad y consistencia.

Ventajas del enfoque “contrato primero”

1. **Interoperabilidad:** Clientes de diferentes lenguajes pueden consumir el servicio siguiendo el contrato WSDL.
2. **Claridad:** Define claramente las operaciones y los datos antes de programar.
3. **Validación automática:** Los mensajes SOAP se pueden validar contra el WSDL.
4. **Menor acoplamiento:** Cambios en la implementación no afectan el contrato mientras se cumpla WSDL.

Por qué las demás opciones son incorrectas

- **A. Se implementa el servicio web y luego se define su contrato.**
✗ Esto corresponde al enfoque “contract-last”, no “contrato primero”.
- **C. No es aplicable en el desarrollo de servicios web SOAP.**
✗ Falso. Es ampliamente utilizado y recomendado en SOAP.
- **D. El enfoque de "contrato primero" no se utiliza en SOAP.**
✗ Incorrecto. SOAP está diseñado para trabajar con WSDL y se beneficia del enfoque “contrato primero”.

El enfoque “**contrato primero**” asegura que el servicio web se desarrolle **siguiendo un contrato WSDL predefinido**, garantizando interoperabilidad, consistencia y claridad en la comunicación entre cliente y servidor.

56. ¿Cuál de las siguientes opciones describe mejor la función del elemento SOAP Body?

- A. **Contiene la información principal del mensaje SOAP.**
- B. Define la dirección de destino del mensaje SOAP.
- C. Proporciona información adicional sobre la solicitud o respuesta SOAP.
- D. Establece las reglas de validación para el mensaje SOAP.

Justificación:

El elemento **<Body>** en SOAP es **obligatorio** y se utiliza para **transportar la información principal del mensaje**, es decir, los datos que representan la operación que se está invocando o la respuesta del servicio.

- Si ocurre un error durante la ejecución del servicio, **<Body>** también puede contener un **elemento <Fault>** que describe el fallo.

Estructura básica de un mensaje SOAP

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <!-- Información opcional, como seguridad o metadatos -->
  </soap:Header>
  <soap:Body>
    <m:getProducto xmlns:m="http://example.com/tienda">
      <m:id>25</m:id>
    </m:getProducto>
  </soap:Body>
</soap:Envelope>
```

- <Body> contiene la **operación principal** (getProducto) y sus datos (id=25).
- <Header> es opcional y contiene información adicional.

Por qué las demás opciones son incorrectas

- **B. Define la dirección de destino del mensaje SOAP.**
✗ Incorrecto. La dirección se define en el **endpoint del WSDL** o en la URL de la solicitud HTTP.
- **C. Proporciona información adicional sobre la solicitud o respuesta SOAP.**
✗ Falso. Esto corresponde al <Header>.
- **D. Establece las reglas de validación para el mensaje SOAP.**
✗ Incorrecto. La validación se realiza mediante **XML Schema** o WSDL, no mediante <Body>.

El **SOAP Body** es el **contenedor obligatorio** que transporta la información principal de la solicitud o respuesta del servicio web, incluyendo los datos de la operación o errores.

57. ¿Cuál de las siguientes afirmaciones es verdadera sobre SOAP y seguridad?

- A. SOAP no es compatible con mecanismos de seguridad, como la encriptación y la autenticación.
- B. SOAP tiene un alto nivel de seguridad incorporado mediante la firma y el cifrado de mensajes.**
- C. SOAP solo se puede utilizar en redes privadas y no es seguro para su uso en Internet.
- D. SOAP no permite la autenticación de usuarios en servicios web.

Justificación:

SOAP proporciona mecanismos de seguridad avanzados mediante la especificación **WS-Security**, que permite:

1. **Firma digital de mensajes**
 - Garantiza la **integridad y autenticidad** del mensaje.
 - Asegura que el mensaje no haya sido alterado y que proviene de un remitente confiable.
2. **Cifrado de mensajes**
 - Protege la **confidencialidad** de los datos durante la transmisión.
 - Solo los destinatarios autorizados pueden descifrar la información.
3. **Autenticación de usuarios**
 - Permite incluir **tokens de seguridad** (UsernameToken, BinarySecurityToken, SAML Token) en el <Header> del mensaje SOAP.
4. **Seguridad independiente del transporte**
 - A diferencia de HTTPS, WS-Security protege el mensaje completo, incluso si pasa por múltiples intermediarios.

Por qué las demás opciones son incorrectas

- **A. SOAP no es compatible con mecanismos de seguridad.**
✗ Falso, SOAP soporta **firma, cifrado y autenticación**.
- **C. SOAP solo se puede utilizar en redes privadas.**
✗ Incorrecto, SOAP es seguro para Internet si se combina con WS-Security o HTTPS.
- **D. SOAP no permite la autenticación de usuarios.**
✗ Falso, SOAP permite autenticación mediante tokens o certificados en el <Header>.

SOAP permite implementar un **alto nivel de seguridad**, incluyendo integridad, confidencialidad y autenticación de usuarios, mediante WS-Security y cifrado de mensajes.

58. ¿Qué es JAX-WS en el contexto de SOAP en Java?

- A. Java API for XML (JAX) - Web Services.
- B. Java Architecture for XML-based Web Services.
- C. Java API for XML Web Services.
- D. Java Advanced XML Web Services.

Justificación:

JAX-WS (Java API for XML Web Services) es la **API oficial de Java** para desarrollar y consumir **servicios web SOAP**. Su función principal es facilitar:

1. **Creación de servicios web SOAP en Java**
 - Permite exponer clases y métodos Java como operaciones SOAP mediante anotaciones (@WebService, @WebMethod).
2. **Consumo de servicios web SOAP**
 - Los clientes pueden generar automáticamente clases proxy a partir de un WSDL, para invocar métodos SOAP sin preocuparse por los detalles de XML.
3. **Integración con WSDL**
 - JAX-WS puede generar automáticamente WSDL desde la implementación de la clase Java o generar clases Java a partir de un WSDL (enfoque “contrato primero”).
4. **Soporte de estándares WS-***
 - Seguridad (WS-Security), confiabilidad (WS-ReliableMessaging) y transacciones.

Ejemplo simple

```
import jakarta.jws.WebService;  
import jakarta.jws.WebMethod;
```

```
@WebService  
public class CalculadoraService {  
    @WebMethod  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
}
```

- Con esta clase y JAX-WS, se puede exponer un servicio SOAP completo con WSDL y soporte para interoperabilidad.

Por qué las demás opciones son incorrectas

- **A. Java API for XML (JAX) - Web Services.**
✗ Incorrecto, el nombre completo es **Java API for XML Web Services**, no JAX-Web Services.
- **B. Java Architecture for XML-based Web Services.**
✗ Falso, JAX-WS es una API, no una arquitectura.
- **D. Java Advanced XML Web Services.**
✗ Incorrecto, no existe una especificación oficial con este nombre.

JAX-WS es la **API estándar de Java** para implementar y consumir servicios web **SOAP**, proporcionando interoperabilidad, integración con WSDL y soporte para estándares de seguridad.

59. ¿Cuál de las siguientes afirmaciones es verdadera sobre la interoperabilidad en SOAP?

- A. SOAP no es compatible con sistemas no basados en Java.
- B. SOAP permite la interoperabilidad entre diferentes plataformas y lenguajes de programación.**
- C. SOAP solo puede interoperar con servicios web RESTful.
- D. La interoperabilidad es un problema común en servicios web SOAP.

Justificación:

SOAP fue diseñado para ser **independiente de la plataforma y del lenguaje de programación**, lo que permite que un servicio web implementado en un lenguaje pueda ser consumido por clientes en otro lenguaje, siempre que ambos cumplan con el estándar SOAP y el contrato WSDL.

Cómo se logra la interoperabilidad

1. **Uso de XML**
 - Los mensajes SOAP están codificados en XML, lo que asegura que cualquier sistema capaz de procesar XML pueda entender el mensaje.
2. **Definición mediante WSDL**
 - WSDL describe las operaciones del servicio, los tipos de datos y la estructura de los mensajes, permitiendo que clientes de diferentes lenguajes generen código automáticamente para consumir el servicio.
3. **Estándares WS-***
 - SOAP soporta estándares como WS-Security, WS-ReliableMessaging, y WS-Addressing, garantizando interoperabilidad incluso con funcionalidades avanzadas.

Por qué las demás opciones son incorrectas

- **A. SOAP no es compatible con sistemas no basados en Java.**
✗ Falso. SOAP funciona en **cualquier lenguaje** que implemente el estándar.
- **C. SOAP solo puede interoperar con servicios web RESTful.**
✗ Incorrecto. REST es un estilo arquitectónico distinto; SOAP no depende de REST.
- **D. La interoperabilidad es un problema común en servicios web SOAP.**
✗ Falso. La interoperabilidad es **una ventaja clave de SOAP**.

SOAP garantiza la **interoperabilidad entre diferentes plataformas y lenguajes de programación**, utilizando XML y WSDL como estándares de comunicación.

60. ¿Cuál de las siguientes opciones describe mejor el enfoque de "contrato primero" en el desarrollo de servicios web SOAP?

- A. Se implementa el servicio web y luego se define su contrato.
- B. Se define el contrato del servicio web antes de su implementación.**
- C. No es aplicable en el desarrollo de servicios web SOAP.
- D. El enfoque de "contrato primero" no se utiliza en SOAP.

Justificación:

El enfoque **"contrato primero" (contract-first)** en SOAP consiste en **definir primero el contrato del servicio web**, generalmente mediante un archivo **WSDL**, antes de desarrollar la lógica del servicio.

El contrato describe:

- Las **operaciones disponibles** en el servicio.
- Los **tipos de datos y parámetros** que se utilizan.
- La **estructura de los mensajes SOAP**.
- La **URL o endpoint** donde se expondrá el servicio.

Una vez definido el WSDL, los desarrolladores implementan la lógica del servicio asegurándose de **cumplir exactamente con el contrato especificado**, lo que garantiza interoperabilidad y consistencia.

Ventajas del enfoque “contrato primero”

1. **Interoperabilidad:** Clientes de diferentes lenguajes pueden consumir el servicio siguiendo el contrato WSDL.
2. **Claridad:** Define claramente las operaciones y los datos antes de programar.
3. **Validación automática:** Los mensajes SOAP se pueden validar contra el WSDL.
4. **Menor acoplamiento:** Cambios en la implementación no afectan el contrato mientras se cumpla WSDL.

Por qué las demás opciones son incorrectas

- **A. Se implementa el servicio web y luego se define su contrato.**
✗ Esto corresponde al enfoque “contract-last”, no “contrato primero”.
- **C. No es aplicable en el desarrollo de servicios web SOAP.**
✗ Falso. Es ampliamente utilizado y recomendado en SOAP.
- **D. El enfoque de "contrato primero" no se utiliza en SOAP.**
✗ Incorrecto. SOAP está diseñado para trabajar con WSDL y se beneficia del enfoque “contrato primero”.

El enfoque “**contrato primero**” asegura que el servicio web se desarrolle **siguiendo un contrato WSDL predefinido**, garantizando interoperabilidad, consistencia y claridad en la comunicación entre cliente y servidor.

61. ¿Cuál de las siguientes opciones describe mejor la función del elemento SOAP Envelope?

- A. Define la estructura y el formato de los mensajes SOAP.
- B. Proporciona información adicional en el cuerpo del mensaje SOAP.
- C. Envuelve el mensaje SOAP completo y lo identifica como tal.**
- D. Contiene la información principal del mensaje SOAP.

Justificación:

El elemento **<Envelope>** es el **contenedor raíz obligatorio** de un mensaje SOAP. Su función principal es **definir los límites del mensaje SOAP**, indicando al receptor que se trata de un mensaje SOAP válido y estructurado.

- Contiene **dos elementos hijos opcionales/obligatorios**:
 1. **<Header>** (opcional): información adicional, como seguridad o metadatos.
 2. **<Body>** (obligatorio): información principal del mensaje o errores (**<Fault>**).

Ejemplo de SOAP Envelope

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <!-- Información opcional -->
  </soap:Header>
  <soap:Body>
    <m:getProducto xmlns:m="http://example.com/tienda">
      <m:id>25</m:id>
    </m:getProducto>
  </soap:Body>
</soap:Envelope>
```

- <Envelope> **envuelve todo el mensaje** y garantiza que sea reconocido como un mensaje SOAP.
- <Body> contiene la información principal, mientras que <Header> contiene datos adicionales opcionales.

Por qué las demás opciones son incorrectas

- **A. Define la estructura y el formato de los mensajes SOAP.**
✗ Parcialmente cierto, pero su función principal es **envolver el mensaje completo**, no definir los detalles internos.
- **B. Proporciona información adicional en el cuerpo del mensaje SOAP.**
✗ Falso. Eso corresponde al <Header>.
- **D. Contiene la información principal del mensaje SOAP.**
✗ Incorrecto. La información principal está en <Body>, no en <Envelope>.

El **SOAP Envelope** es el **contenedor raíz** que envuelve todo el mensaje SOAP y lo identifica claramente como un mensaje SOAP válido.

- 62. ¿Cuál de las siguientes opciones describe mejor el uso de SOAP en comparación con REST en términos de complejidad?**
- A. SOAP es más simple y liviano que REST.
 - B. SOAP es más complejo y pesado que REST.**
 - C. SOAP y REST tienen la misma complejidad.
 - D. La complejidad depende del lenguaje de programación utilizado.

Justificación:

SOAP y REST son dos enfoques distintos para servicios web, y se diferencian notablemente en **complejidad y peso**:

Comparación SOAP vs REST

Característica	SOAP	REST
Formato de mensaje	XML estricto y detallado	JSON, XML u otros formatos ligeros
Estándares y protocolos	WS-Security, WS-ReliableMessaging, WS-Addressing	HTTP estándar (GET, POST, PUT, DELETE)
Transporte	HTTP, SMTP, JMS, etc.	Principalmente HTTP
Implementación	Rígida; requiere WSDL, generación de clases y validación de mensajes	Flexible; fácil de implementar y consumir
Uso típico	Integraciones empresariales complejas, transacciones seguras	Aplicaciones web y móviles ligeras

Conclusión clave

- SOAP es más **pesado** debido al uso obligatorio de XML y a la implementación de estándares adicionales.
- REST es más **ligero y simple**, ideal para aplicaciones web y móviles que requieren rapidez y eficiencia.

Por qué las demás opciones son incorrectas

- **A. SOAP es más simple y liviano que REST.**
✗ Falso, es lo contrario.

- **C. SOAP y REST tienen la misma complejidad.**
✗ Incorrecto, SOAP es más estructurado y pesado.
- **D. La complejidad depende del lenguaje de programación.**
✗ La complejidad depende del protocolo y los estándares, no del lenguaje.

SOAP es **más complejo y pesado** que REST, especialmente en entornos empresariales que requieren transacciones, seguridad y estándares estrictos.

63. ¿Cuál de las siguientes opciones describe mejor el propósito del elemento SOAP Fault?

- A. Proporcionar información adicional sobre el estado de un servicio web.
- B. Indicar que se produjo un error durante la ejecución del servicio.**
- C. Establecer la dirección de destino del mensaje SOAP.
- D. Controlar el flujo de ejecución del servicio.

Justificación:

El elemento **<Fault>** de SOAP se utiliza para **informar sobre errores ocurridos durante el procesamiento de un mensaje SOAP**. Se encuentra dentro del **<Body>** y permite que el cliente reciba detalles claros sobre fallos en la operación solicitada.

Estructura básica de un SOAP Fault

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">  
  <soap:Body>  
    <soap:Fault>  
      <faultcode>soap:Client</faultcode>  
      <faultstring>Error en la solicitud: parámetro inválido</faultstring>  
      <detail>  
        <m:InvalidParameter xmlns:m="http://example.com/tienda">  
          El parámetro "id" no existe.  
        </m:InvalidParameter>  
      </detail>  
    </soap:Fault>  
  </soap:Body>  
</soap:Envelope>
```

- **<faultcode>**: Código de error (por ejemplo, Client o Server).
- **<faultstring>**: Mensaje descriptivo del error.
- **<detail>**: Información opcional adicional sobre el fallo.

Por qué las demás opciones son incorrectas

- **A. Proporcionar información adicional sobre el estado de un servicio web.**
✗ Parcialmente cierto, pero el **<Fault>** se usa específicamente **para errores**, no para cualquier estado.
- **C. Establecer la dirección de destino del mensaje SOAP.**
✗ Incorrecto. La dirección se define en el endpoint del WSDL.
- **D. Controlar el flujo de ejecución del servicio.**
✗ Falso. SOAP Fault no controla el flujo, solo reporta errores.

El **SOAP Fault** permite **notificar errores** ocurridos durante la ejecución del servicio, proporcionando códigos y mensajes que ayudan al cliente a entender y manejar la falla.